

Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут» ім. Ігоря Сікорського

Розрахунково-графічна робота

з дисципліни “Бази даних та засоби управління”

«Створення застосунку бази даних, орієнтованого на взаємодію з СУБД PostgreSQL»

Виконав студент групи: КВ-41
Бабін Андрій

Київ 2026

Мета роботи

Набути навичок розроблення консольного застосунку на Python 3, який взаємодіє з СУБД PostgreSQL за допомогою параметризованих SQL-запитів (Psycopg 3, без ORM), коректно обробляє помилки бази даних і має структуру за шаблоном MVC.

Завдання

Для таблиць бази даних, створеної в лабораторній роботі №1, потрібно було:

- реалізувати перегляд, додавання, редагування та видалення даних у консолі;
- реалізувати генерацію випадкових даних SQL-запитом на стороні PostgreSQL;
- реалізувати не менше трьох пошукових запитів до двох і більше таблиць із фільтрами, які вводить користувач: діапазони для чисел і дат, LIKE для рядків, True/False для логічних полів;
- організувати код за шаблоном MVC.

Додаткові вимоги: цілісність даних контролюється на рівні бази даних, а користувач бачить зрозумілі повідомлення замість системних помилок; усі значення передаються в запити параметрами, а не підставляються в текст запиту; для перевірки швидкодії потрібно згенерувати 100 000 записів; після кожного пошуку виводиться час виконання в мілісекундах.

Предметна галузь - Сайт знайомств

Робота зроблена на базі даних сайту знайомств з лабораторної роботи №1.

Посилання на репозиторій

https://github.com/AndrewKPIKB/rgr_db

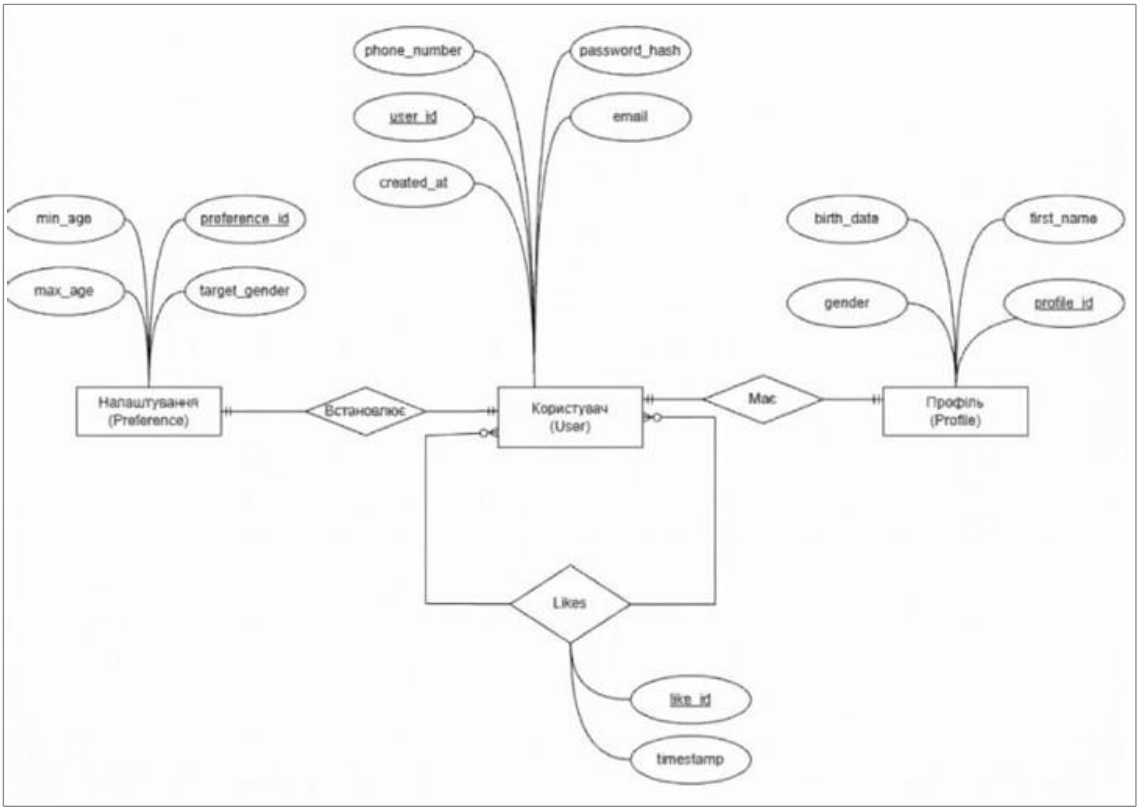
База даних

Для сайту знайомств у базі є чотири таблиці:

- User - дані користувача для входу: email, номер телефону, хеш пароля, дата реєстрації;
- Profile - додаткові дані: ім'я, дата народження, стать;
- Preference - що користувач шукає: бажана стать і вік від та до;
- Like - вподобайки: хто кого вподобав і коли.

Один користувач має один профіль і одні налаштування (зв'язок 1:1). Лайки зв'язують двох користувачів через два зовнішні ключі: user_id_from (хто поставив) і user_id_to (кому поставили).

Під час виконання РГР до таблиці User додали ще одне поле is_verified (boolean, за замовчуванням false). Воно потрібне, щоб був пошук за логічним значенням.



ER-діаграма бази даних (нотація Пітера Чена)

На діаграмі ще немає поля is_verified, бо його додали вже після лабораторної роботи.

Таблиця	**Стовпці**	**Обмеження**
`User`	user_id, email, password_hash, phone_number, created_at, is_verified	PK user_id; UNIQUE для email і phone_number; перевірка формату email; усі поля NOT NULL
`Profile`	profile_id, user_id, first_name, birth_date, gender	PK profile_id; user_id UNIQUE і FK на User; стать тільки male, female або other; дата народження після 1900 року
`Preference`	preference_id, user_id, target_gender, min_age, max_age	PK preference_id; user_id UNIQUE і FK на User; min_age не менше 18, max_age не більше 120, min_age не більший за max_age
`Like`	like_id, user_id_from, user_id_to, timestamp	PK like_id; два FK на User; пара (from, to) UNIQUE; не можна лайкнути самого себе

Усі зовнішні ключі створені з ON DELETE CASCADE. Повний SQL створення таблиць є в додатку.

Нижче структура схеми в pgAdmin та список обмежень, які задані в самій базі даних (p - PRIMARY KEY, f - FOREIGN KEY, u - UNIQUE, c - CHECK, n - NOT NULL).

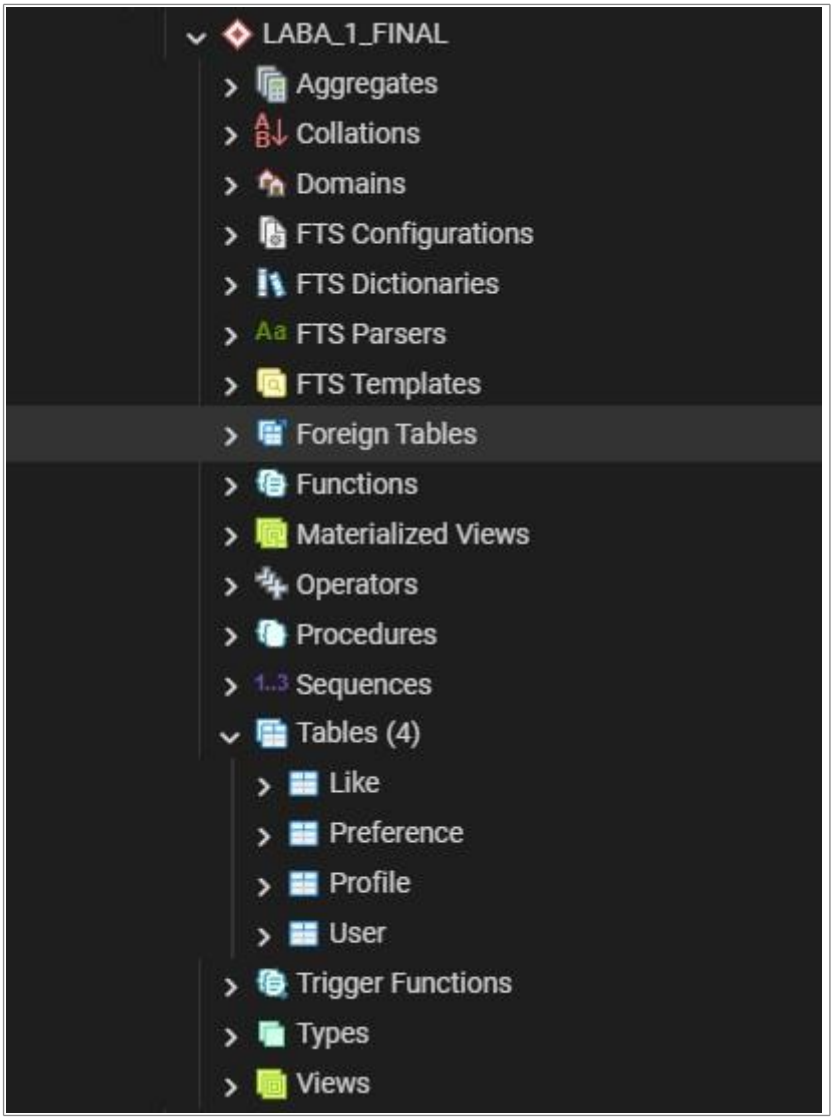


Схема LABA_1_FINAL у pgAdmin

Query | Query History

```
1  SELECT conrelid::regclass AS table_name, conname, contype
2  FROM pg_constraint
3  WHERE connamespace = '"LABA_1_FINAL"'::regnamespace
4  ORDER BY 1, 3;
```

Запит, який виводить обмеження таблиць

Data Output | Messages | Notifications

SQL

	table_name regclass	conname name	contype "char"
1	"User"	user_email_ck	c
2	"User"	User_is_verified_not_null	n
3	"User"	User_user_id_not_null	n
4	"User"	User_email_not_null	n
5	"User"	User_password_hash_not_null	n
6	"User"	User_phone_number_not_null	n
7	"User"	User_created_at_not_null	n
8	"User"	User_pkey	p
9	"User"	user_phone_uq	u
10	"User"	user_email_uq	u
11	"Profile"	profile_gender_ck	c
12	"Profile"	profile_birth_ck	c
13	"Profile"	profile_user_fk	f
14	"Profile"	Profile_user_id_not_null	n
15	"Profile"	Profile_first_name_not_null	n
16	"Profile"	Profile_birth_date_not_null	n
17	"Profile"	Profile_gender_not_null	n
18	"Profile"	Profile_profile_id_not_null	n
19	"Profile"	Profile_pkey	p
20	"Profile"	Profile_user_id_key	u
21	"Preference"	pref_max_age_ck	c
22	"Preference"	pref_range_ck	c
23	"Preference"	pref_min_age_ck	c
24	"Preference"	pref_gender_ck	c
25	"Preference"	preference_user_fk	f
26	"Preference"	Preference_preference_id_not_...	n
27	"Preference"	Preference_user_id_not_null	n
28	"Preference"	Preference_pkey	p
29	"Preference"	Preference_user_id_key	u
30	"Like"	like_self_ck	c
31	"Like"	like_from_fk	f
32	"Like"	like_to_fk	f
33	"Like"	Like_user_id_from_not_null	n
34	"Like"	Like_timestamp_not_null	n
35	"Like"	Like_user_id_to_not_null	n

Обмеження таблиць User, Profile, Preference

35	"Like"	Like_user_id_to_not_null	n
36	"Like"	Like_like_id_not_null	n
37	"Like"	Like_pkey	p
38	"Like"	like_pair_uq	u

Обмеження таблиці Like

Використані засоби та структура програми

Що	**Яке**
Мова програмування	Python 3.12
СУБД	PostgreSQL 18
Бібліотека для роботи з БД	Psycopg 3 (psycopg[binary] 3.3.6), без ORM
Читання налаштувань	python-dotenv 1.2.4
Робота з SQL	pgAdmin 4
Середовище розробки	Visual Studio Code
Інтерфейс	консоль

Структура проєкту:

```
rgr_db/
├── main.py                  # запуск програми
├── requirements.txt
├── .env.example             # приклад налаштувань (без пароля)
├── sql/                     # SQL-скрипти: схема, генерація, перевірки, індекси
├── model/                   # робота з PostgreSQL
│   ├── db.py  tables.py  crud.py  generator.py  search.py  errors.py
├── view/
│   └── console.py           # меню та виведення
└── controller/              # сценарії та перевірка введення
    ├── main_controller.py  app.py  generation_controller.py
    └── search_controller.py  inputs.py
```

Дані для підключення (хост, порт, назва бази, користувач, пароль, схема) програма бере з файлу .env. У репозиторії цього файлу немає, він додається в .gitignore. Замість нього лежить .env.example без пароля.

Схема меню програми:

```
Головне меню
├── 1. Перегляд, внесення, редагування, вилучення даних
│       └── Користувачі | Профілі | Вподобання | Лайки
│               └── Переглянути | Додати | Редагувати | Видалити
├── 2. Генерація даних (SQL)
│       └── Користувачі | Профілі | Налаштування | Лайки | Повний набір
├── 3. Пошук
│       └── 1) за профілем  2) за лайками  3) кандидати  4) за налаштуваннями
└── 0. Вихід
```

Пункт №1. Перегляд, внесення, редагування та видалення даних

Як це зроблено

Усі запити для роботи з таблицями лежать у model/crud.py. Значення від користувача передаються окремо від тексту запиту, у cur.execute(query, params). Назви таблиць і стовпців підставляються тільки через psycopg.sql.Identifier, тому SQL ніде не склеюється з введених рядків.

```
def insert_row(conn, cfg, values: dict):
    query = sql.SQL("INSERT INTO {t} ({cols}) VALUES ({ph}) RETURNING {pk}").format(
        t=sql.Identifier(cfg["table"]),
        cols=sql.SQL(", ").join(sql.Identifier(c) for c in values),
        ph=sql.SQL(", ").join(sql.Placeholder() for _ in values),
        pk=sql.Identifier(cfg["pk"]))
    result, _ = _execute(conn, query, list(values.values()), op="insert", fetch=True)
```

Спочатку введення перевіряє контролер (controller/inputs.py): чи це UUID, дата, true/false або число. Якщо ні, запит до бази навіть не відправляється. Усе інше перевіряє сама база даних, а модуль model/errors.py за назвою порушеного обмеження показує повідомлення українською, тому системний текст помилки користувач не бачить.

```
def parse(kind, text):
    try:
        if kind == "int":
            return int(text)
        if kind == "date":
            return date.fromisoformat(text)
        if kind == "uuid":
            return uuid.UUID(text)
        if kind == "bool":
            value = text.strip().lower()
            if value in _TRUE:
                return True
            if value in _FALSE:
                return False
            raise ValueError("не булеве значення")
    except ValueError:
        raise ValueError({"int": "потрібно ввести ціле число",
                          "date": "дата має бути у форматі РРРР-ММ-ДД",
                          "uuid": "некоректний UUID",
                          "bool": "введіть true або false"}[kind])
    return text

CONSTRAINT_MESSAGES = {
    "user_email_uq": "Користувач з таким email уже існує.",
    "user_phone_uq": "Цей номер телефону вже використовується.",
    "user_email_ck": "Некоректний формат email.",
    "Profile_user_id_key": "Для цього користувача профіль уже існує.",
    "profile_gender_ck": "Стать має бути: male, female або other.",
    "pref_range_ck": "Мінімальний вік не може бути більшим за максимальний.",
    "like_pair_uq": "Такий лайк уже існує.",
    "like_self_ck": "Не можна поставити лайк самому собі.",
    # ... решта обмежень
}

def translate(e, op=""):
    name = getattr(getattr(e, "diag", None), "constraint_name", None)
    if name in CONSTRAINT_MESSAGES:
        return CONSTRAINT_MESSAGES[name]
    if isinstance(e, (pg.ForeignKeyViolation, pg.RestrictViolation)):
        if op == "delete":
            return "Не можна видалити: на цей запис посилаються інші записи."
        return "Вказаного користувача не існує (порушено зовнішній ключ)."
    ...
```

Додавання і перегляд

Спочатку додав двох користувачів: Anna (підтверджена, true) і Ivan (підтвердження не вводив, тому false). Потім для кожного додав профіль і вподобання, а також лайки один одному.

```
===== РГР: ОБЕРІТЬ ПУНКТ ЗАВДАННЯ =====
1. Пункт 1 - перегляд, внесення, редагування, вилучення даних
2. Пункт 2 - генерація даних (SQL)
3. Пункт 3 - пошук (параметризовані запити)
0. Вихід
Ваш вибір: 1

===== ГОЛОВНЕ МЕНЮ =====
1. Користувачі (User)
2. Профілі (Profile)
3. Вподобання (Preference)
4. Лайки (Like)
0. Назад до вибору пункту
Ваш вибір: 1
```

Головне меню і меню вибору таблиці

```
===== ГОЛОВНЕ МЕНЮ =====
1. Користувачі (User)
2. Профілі (Profile)
3. Вподобання (Preference)
4. Лайки (Like)
0. Назад до вибору пункту
Ваш вибір: 1

--- Користувачі (User) ---
1. Переглянути
2. Додати
3. Редагувати
4. Видалити
0. Назад
Ваш вибір: 2
Email: anna@mail.com
Хеш пароля: hash_anna
Телефон: +380501111111
Підтверджений true/false (Enter - false): true
Дата створення РРРР-ММ-ДД (Enter - сьогодні): 2007-03-29
Додано, id = a756a035-faf2-4f46-a629-797937ead8b2
```

Додавання користувача anna@mail.com (підтверджений true, дата 2007-03-29)

```
--- Профілі (Profile) ---
1. Переглянути
2. Додати
3. Редагувати
4. Видалити
0. Назад
Ваш вибір: 2
ID користувача (UUID): 8d4b8288-66e9-4131-8859-1331e4e4f1fc
Ім'я: Ivan
Дата народження РРРР-ММ-ДД: 1998-03-15
Стать (male/female/other): male
Додано, id = c097dbc7-bf6e-4e85-8798-6d49ca1ed656

--- Профілі (Profile) ---
1. Переглянути
2. Додати
3. Редагувати
4. Видалити
0. Назад
Ваш вибір: 2
ID користувача (UUID): a756a035-faf2-4f46-a629-797937ead8b2
Ім'я: Anna
Дата народження РРРР-ММ-ДД: 2000-05-10
Стать (male/female/other): female
Додано, id = 64f729dd-9d83-4720-b89b-773eaedd637e
```

Додавання профілів: спочатку Ivan, потім Anna

```
--- Вподобання (Preference) ---
1. Переглянути
2. Додати
3. Редагувати
4. Видалити
0. Назад
Ваш вибір: 2
ID користувача (UUID): a756a035-faf2-4f46-a629-797937ead8b2
Бажана стать (Enter - пропустити): male
Мін. вік (Enter - пропустити): 20
Макс. вік (Enter - пропустити): 30
Додано, id = d7fabc98-1806-4944-891a-b99b1fa1497c

--- Вподобання (Preference) ---
1. Переглянути
2. Додати
3. Редагувати
4. Видалити
0. Назад
Ваш вибір: 2
ID користувача (UUID): 8d4b8288-66e9-4131-8859-1331e4e4f1fc
Бажана стать (Enter - пропустити): female
Мін. вік (Enter - пропустити): 20
Макс. вік (Enter - пропустити): 30
Додано, id = 76c3bf00-9dd2-442a-a6ac-5a6c36712595
```

Додавання вподобань: спочатку Anna, потім Ivan

```
--- Лайки (Like) ---
1. Переглянути
2. Додати
3. Редагувати
4. Видалити
0. Назад
Ваш вибір: 2
ID того, хто ставить лайк (UUID):  a756a035-faf2-4f46-a629-797937ead8b2
ID того, кому лайк (UUID): 8d4b8288-66e9-4131-8859-1331e4e4f1fc
Дата РРРР-ММ-ДД (Enter - сьогодні):
Додано, id = 2aa83f0e-f553-4fb7-b2e6-dc6459433a5e

--- Лайки (Like) ---
1. Переглянути
2. Додати
3. Редагувати
4. Видалити
0. Назад
Ваш вибір: 2
ID того, хто ставить лайк (UUID): 8d4b8288-66e9-4131-8859-1331e4e4f1fc
ID того, кому лайк (UUID): a756a035-faf2-4f46-a629-797937ead8b2
Дата РРРР-ММ-ДД (Enter - сьогодні):
Додано, id = 18cd6760-1367-4a38-a4c6-1600a90028b1
```

Додавання лайків: від Anna до Ivan і від Ivan до Anna

Query Query History

```
1  SELECT * FROM "LABA_1_FINAL"."User";
```

Data Output Messages Notifications

Showing rows: 1 to 2

	user_id [PK] uuid	email character varying (255)	password_hash character varying (255)	phone_number character varying (30)	created_at date	is_verified boolean
1	a756a035-faf2-4f46-a629-797937ead8...	anna@mail.com	hash_anna	+380501111111	2007-03-29	true
2	8d4b8288-66e9-4131-8859-1331e4e4f...	ivan@mail.com	hash_ivan	+380502222222	2026-11-08	false

Таблиця User у pgAdmin: Anna з is_verified = true, Ivan з false

Query Query History

```
1  SELECT * FROM "LABA_1_FINAL"."Profile";
```

Data Output Messages Notifications

Showing rows: 1 to 2

	profile_id [PK] uuid	user_id uuid	first_name character varying (100)	birth_date date	gender character varying (10)
1	c097dbc7-bf6e-4e85-8798-6d49ca1ed6...	8d4b8288-66e9-4131-8859-1331e4e4f...	Ivan	1998-03-15	male
2	64f729dd-9d83-4720-b89b-773eaedd63...	a756a035-faf2-4f46-a629-797937ead8...	Anna	2000-05-10	female

Таблиця Profile у pgAdmin

Query Query History

```
1  SELECT * FROM "LABA_1_FINAL"."Preference";
```

Data Output Messages Notifications

Showing rows: 1 to

	preference_id [PK] uuid	user_id uuid	target_gender character varying (10)	min_age integer	max_age integer
1	d7fabc98-1806-4944-891a-b99b1fa149...	a756a035-faf2-4f46-a629-797937ead8...	male	20	30
2	76c3bf00-9dd2-442a-a6ac-5a6c367125...	8d4b8288-66e9-4131-8859-1331e4e4f...	female	20	30

Таблиця Preference у pgAdmin

Query Query History

```
1  SELECT * FROM "LABA_1_FINAL"."Like";
```

Data Output Messages Notifications

Showing rows: 1 to 2

	like_id [PK] uuid	user_id_to uuid	user_id_from uuid	timestamp date
1	2aa83f0e-f553-4fb7-b2e6-dc6459433a5e	8d4b8288-66e9-4131-8859-1331e4e4f...	a756a035-faf2-4f46-a629-797937ead8...	2026-11-08
2	18cd6760-1367-4a38-a4c6-1600a90028...	a756a035-faf2-4f46-a629-797937ead8...	8d4b8288-66e9-4131-8859-1331e4e4f...	2026-11-08

Таблиця Like у pgAdmin

```
--- Користувачі (User) ---
1. Переглянути
2. Додати
3. Редагувати
4. Видалити
0. Назад
Ваш вибір: 1
user_id                              | email          | password_hash | phone_number  | created_at | is_verified
-------------------------------------------------------------------------------------------------------------
8d4b8288-66e9-4131-8859-1331e4e4f1fc | ivan@mail.com  | hash_ivan     | +380502222222 | 2026-11-08 | False
a756a035-faf2-4f46-a629-797937ead8b2 | anna@mail.com  | hash_anna     | +380501111111 | 2007-03-29 | True
```

Перегляд таблиці User у програмі: Anna True, Ivan False

```
--- Профілі (Profile) ---
1. Переглянути
2. Додати
3. Редагувати
4. Видалити
0. Назад
Ваш вибір: 1
profile_id                           | user_id                              | first_name | birth_date | gender
-------------------------------------------------------------------------------------------------------------
64f729dd-9d83-4720-b89b-773eaedd637e | a756a035-faf2-4f46-a629-797937ead8b2 | Anna       | 2000-05-10 | female
c097dbc7-bf6e-4e85-8798-6d49ca1ed656 | 8d4b8288-66e9-4131-8859-1331e4e4f1fc | Ivan       | 1998-03-15 | male
```

Перегляд таблиці Profile у програмі

```
--- Вподобання (Preference) ---
1. Переглянути
2. Додати
3. Редагувати
4. Видалити
0. Назад
Ваш вибір: 1
preference_id                        | user_id                              | target_gender | min_age | max_age
-------------------------------------------------------------------------------------------------------------
76c3bf00-9dd2-442a-a6ac-5a6c36712595 | 8d4b8288-66e9-4131-8859-1331e4e4f1fc | female        | 20      | 30
d7fabc98-1806-4944-891a-b99b1fa1497c | a756a035-faf2-4f46-a629-797937ead8b2 | male          | 20      | 30
```

Перегляд таблиці Preference у програмі

```
--- Лайки (Like) ---
1. Переглянути
2. Додати
3. Редагувати
4. Видалити
0. Назад
Ваш вибір: 1
like_id                              | user_id_to                           | user_id_from                         | timestamp
-------------------------------------------------------------------------------------------------------------------------
18cd6760-1367-4a38-a4c6-1600a90028b1 | a756a035-faf2-4f46-a629-797937ead8b2 | 8d4b8288-66e9-4131-8859-1331e4e4f1fc | 2026-11-08
2aa83f0e-f553-4fb7-b2e6-dc6459433a5e | 8d4b8288-66e9-4131-8859-1331e4e4f1fc | a756a035-faf2-4f46-a629-797937ead8b2 | 2026-11-08
```

Перегляд таблиці Like у програмі

```
--- Користувачі (User) ---
1. Переглянути
2. Додати
3. Редагувати
4. Видалити
0. Назад
Ваш вибір: 2
Email: petro@mail.com
Хеш пароля: hash_petro
Телефон: +380504444555
Підтверджений true/false (Enter - false):
Дата створення РРРР-ММ-ДД (Enter - сьогодні):
Додано, id = 4f105dda-5e85-4b8d-93dd-3027d0292f35
```

Додавання користувача petro@mail.com. Підтвердження і дату не вводив, тому false і поточна дата

Query | Query History

```
1  SELECT * FROM "LABA_1_FINAL"."User";
```

Data Output | Messages | Notifications

Showing rows: 1 to 3

	user_id [PK] uuid	email character varying (255)	password_hash character varying (255)	phone_number character varying (30)	created_at date	is_verified boolean
1	8d4b8288-66e9-4131-8859-1331e4e4f...	ivan@mail.com	hash_ivan	+380502222222	2026-11-08	false
2	a756a035-faf2-4f46-a629-797937ead8...	anna@mail.com	hash_anna	+380503333444	2007-03-29	true
3	4f105dda-5e85-4b8d-93dd-3027d0292f...	petro@mail.com	hash_petro	+380504444555	2026-11-08	false

Таблиця User у pgAdmin: Ivan, Anna і Petro

Редагування

При редагуванні вводиться id запису, а потім нові значення. Якщо поле залишити порожнім, воно не змінюється. Це стосується і поля «Підтверджений»: значення true або false, яке було раніше, залишається. При додаванні нового користувача порожнє поле «Підтверджений» дає false. Для цього в read_values є окрема перевірка, а підказка в режимі редагування змінюється на «Enter - без змін»:

```
def read_values(cfg, partial=False):
    values = {}
    for col, label, kind, required in cfg["fields"]:
        if partial:
            # при редагуванні підказка завжди «без змін»
            label = re.sub(r"\(Enter[^)]*\)", "(Enter - без змін)", label)
        text = v.ask(f"{label}: ")
        if text == "":
            if kind == "bool" and not partial:
                values[col] = False          # при додаванні Enter означає false
                continue
            if required and not partial:
                raise ValueError(f"поле «{col}» не може бути порожнім")
            continue                         # при редагуванні поле не змінюється
        values[col] = parse(kind, text)
    return values
```

```
--- Користувачі (User) ---
1. Переглянути
2. Додати
3. Редагувати
4. Видалити
0. Назад
Ваш вибір: 3
ID запису: a756a035-faf2-4f46-a629-797937ead8b2
Введіть нові значення (Enter - залишити без змін):
Email:
Хеш пароля:
Телефон: +380503333444
Підтверджений true/false (Enter - без змін):
Дата створення РРРР-ММ-ДД (Enter - без змін):
Оновлено.
```

Редагування користувача Anna: змінений тільки телефон. В полі «Підтверджений» нічого не вводив, підказка «Enter - без змін»

Query Query History

```
1  SELECT * FROM "LABA_1_FINAL"."User" WHERE email = 'anna@mail.com';
```

Data Output Messages Notifications

Showing rows: 1 to 1

	user_id [PK] uuid	email character varying (255)	password_hash character varying (255)	phone_number character varying (30)	created_at date	is_verified boolean
1	a756a035-faf2-4f46-a629-797937ead8...	anna@mail.com	hash_anna	+380503333444	2007-03-29	true

Результат у pgAdmin: телефон змінився, is_verified лишився true, решта полів та сама

```
--- Користувачі (User) ---
1. Переглянути
2. Додати
3. Редагувати
4. Видалити
0. Назад
Ваш вибір: 3
ID запису: 11111111-1111-1111-1111-111111111111
Введіть нові значення (Enter - залишити без змін):
Email:
Хеш пароля:
Телефон: +380509999999
Підтверджений true/false (Enter - без змін):
Дата створення РРРР-ММ-ДД (Enter - без змін):
Запис з таким id не знайдено.
```

Спроба редагувати запис, якого немає

```
--- Профілі (Profile) ---
1. Переглянути
2. Додати
3. Редагувати
4. Видалити
0. Назад
Ваш вибір: 3
ID запису: 64f729dd-9d83-4720-b89b-773eaedd637e
Введіть нові значення (Enter - залишити без змін):
ID користувача (UUID):
Ім'я: Anna-Maria
Дата народження РРРР-ММ-ДД:
Стать (male/female/other):
Оновлено.
```

Редагування профілю: змінено тільки ім'я

Query | Query History

```
1  SELECT * FROM "LABA_1_FINAL"."Profile";
```

Data Output | Messages | Notifications

Showing rows: 1 to 2

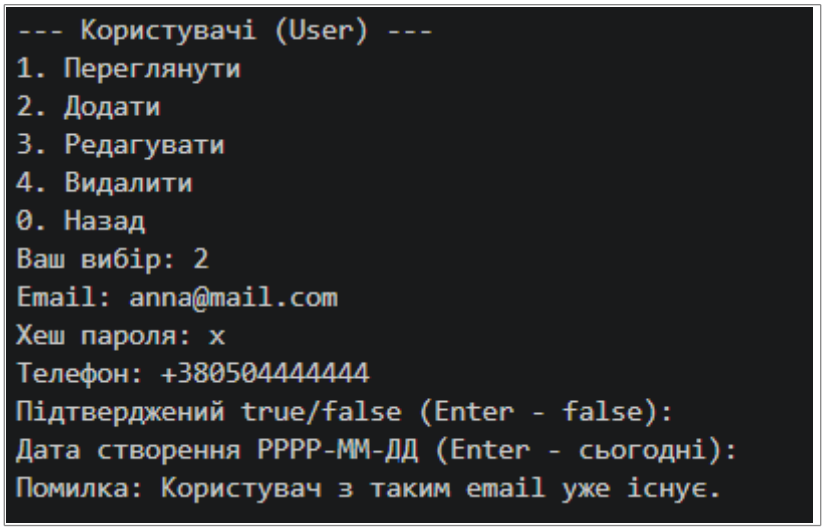
	profile_id [PK] uuid	user_id uuid	first_name character varying (100)	birth_date date	gender character varying (10)
1	c097dbc7-bf6e-4e85-8798-6d49ca1ed6...	8d4b8288-66e9-4131-8859-1331e4e4f...	Ivan	1998-03-15	male
2	64f729dd-9d83-4720-b89b-773eaedd63...	a756a035-faf2-4f46-a629-797937ead8...	Anna-Maria	2000-05-10	female

Таблиця Profile у pgAdmin після редагування: у Anna тепер ім'я Anna-Maria

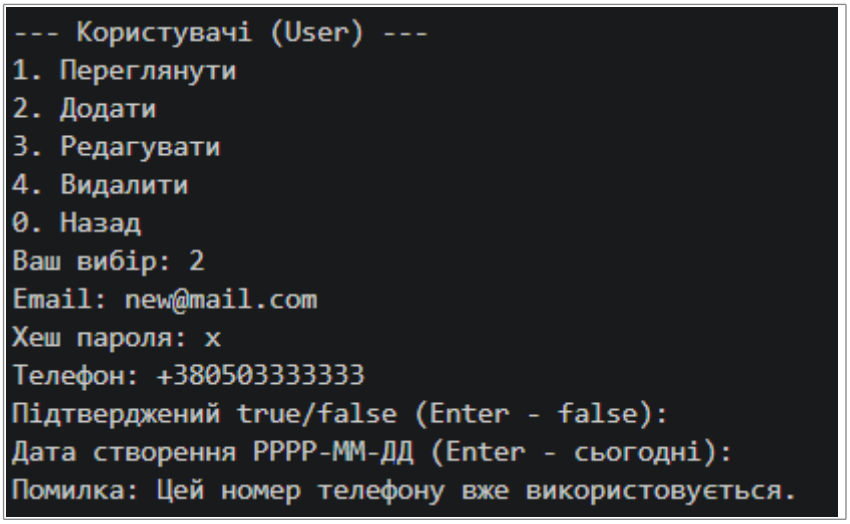
Помилки при введенні даних

Частину помилок знаходить сама програма до звернення до бази (порожнє поле, неправильна дата, UUID, true/false). Решту відхиляє база даних, а програма показує зрозуміле повідомлення.

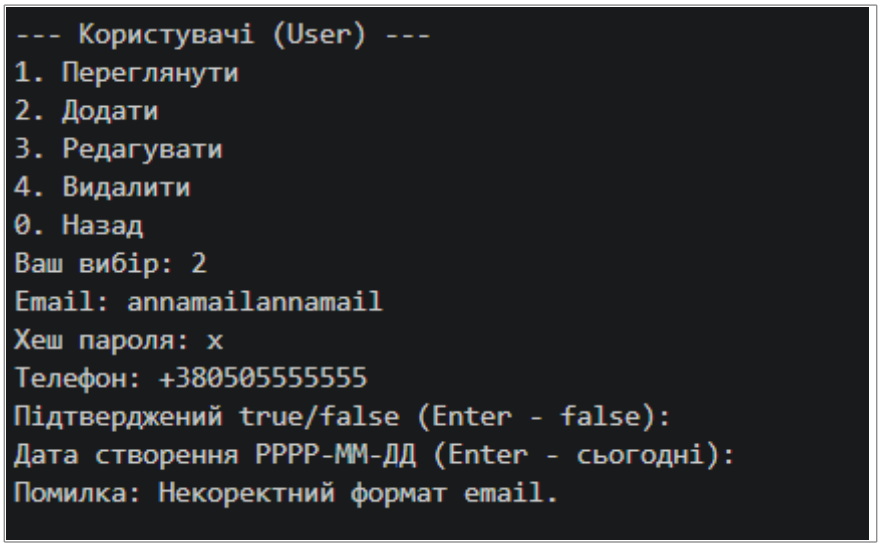
Що введено	Повідомлення програми	Хто перевіряє
email, який уже є	Користувач з таким email уже існує.	БД, UNIQUE user_email_uq
телефон, який уже є	Цей номер телефону вже використовується.	БД, UNIQUE user_phone_uq
email без @	Некоректний формат email.	БД, CHECK user_email_ck
порожній email	Помилка введення: поле «email» не може бути порожнім	програма
abc замість true/false	Помилка введення: введіть true або false	програма
дата 2000/13/45	Помилка введення: дата має бути у форматі РРРР-ММ-ДД	програма
UUID 123	Помилка введення: некоректний UUID	програма
стать abc	Стать має бути: male, female або other.	БД, CHECK profile_gender_ck
другий профіль для користувача	Для цього користувача профіль уже існує.	БД, UNIQUE Profile_user_id_key
профіль для неіснуючого користувача	Вказаного користувача не існує (порушено зовнішній ключ).	БД, FOREIGN KEY profile_user_fk
мінімальний вік більший за максимальний	Мінімальний вік не може бути більшим за максимальний.	БД, CHECK pref_range_ck
мінімальний вік менший за 18	Мінімальний вік не може бути меншим за 18.	БД, CHECK pref_min_age_ck
бажана стать abc	Бажана стать має бути: male, female або other.	БД, CHECK pref_gender_ck
лайк самому собі	Не можна поставити лайк самому собі.	БД, CHECK like_self_ck
повторний лайк	Такий лайк уже існує.	БД, UNIQUE like_pair_uq



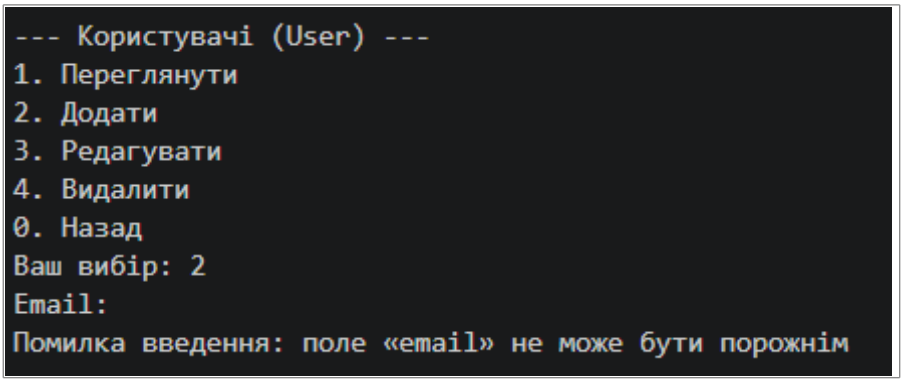
email уже існує



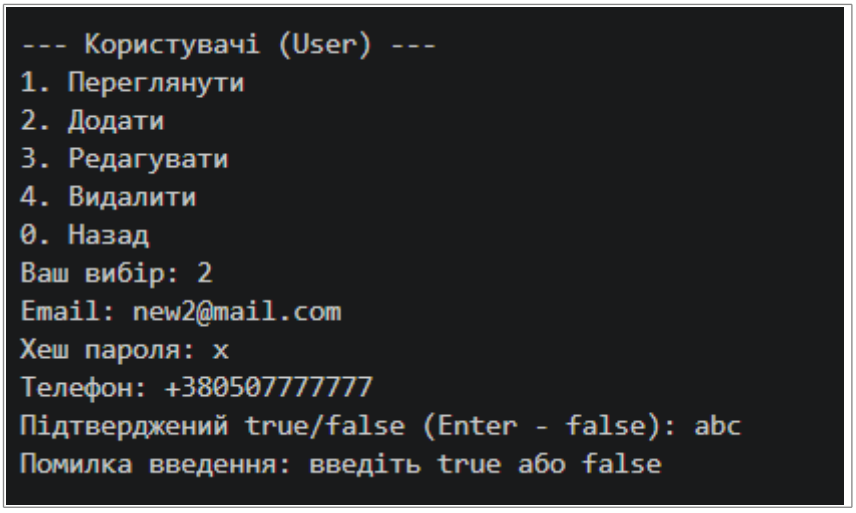
Номер телефону вже використовується



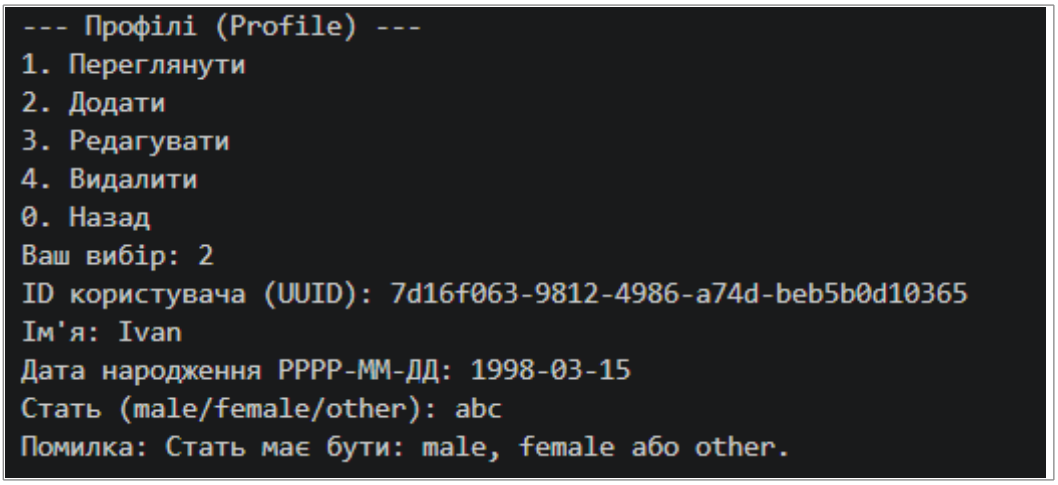
Некоректний формат email



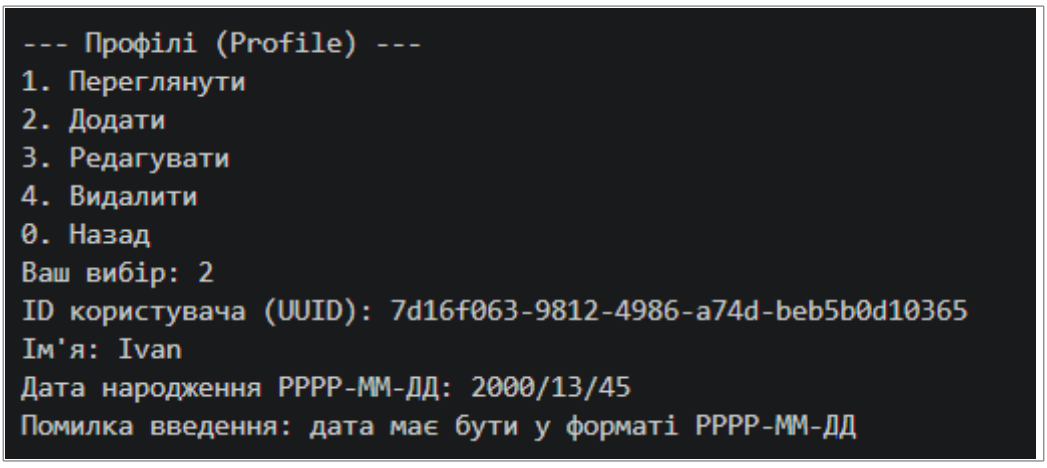
Порожній email



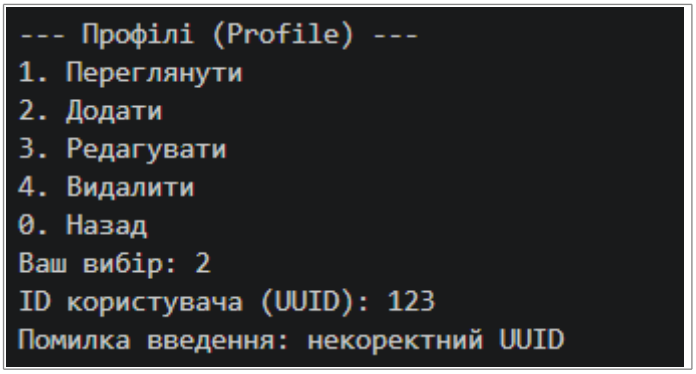
abc замість true/false



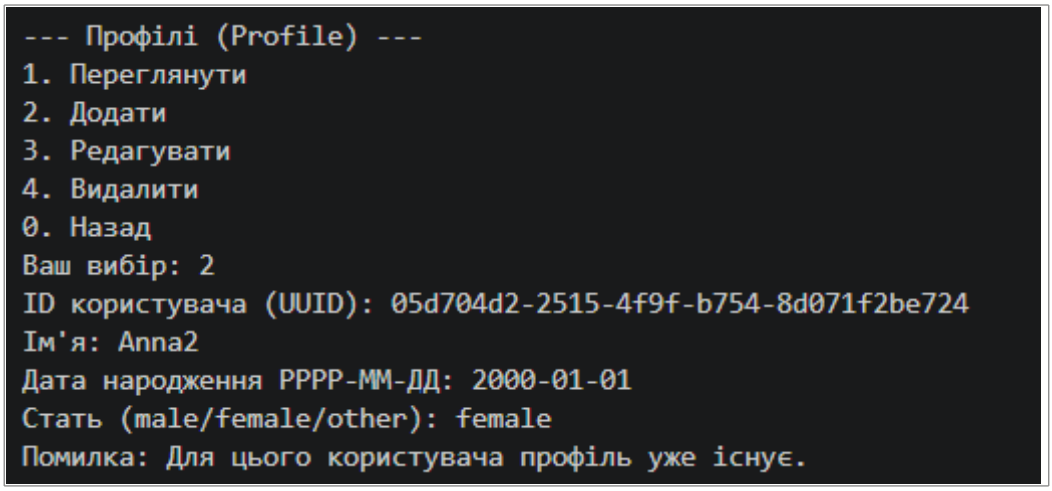
Недопустима стать у профілі



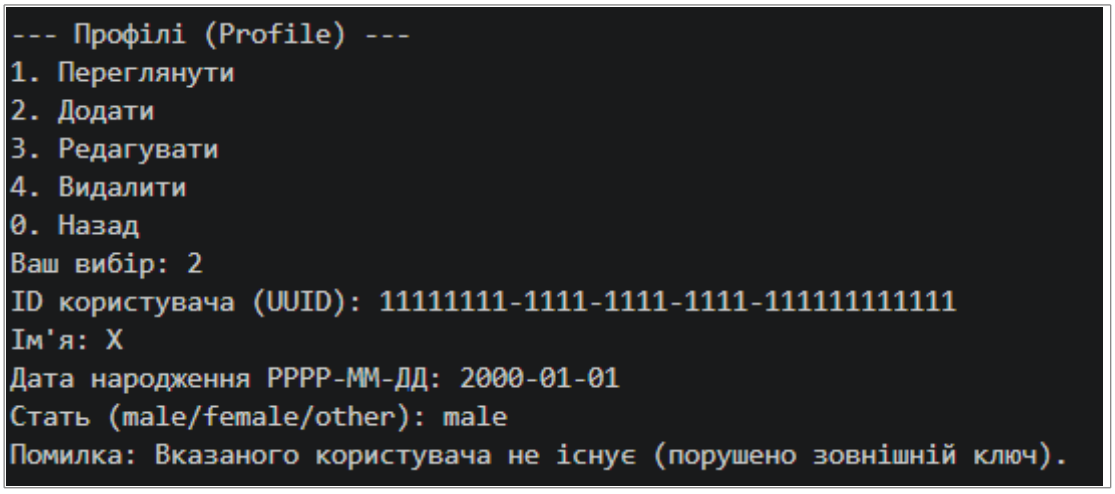
Неправильний формат дати



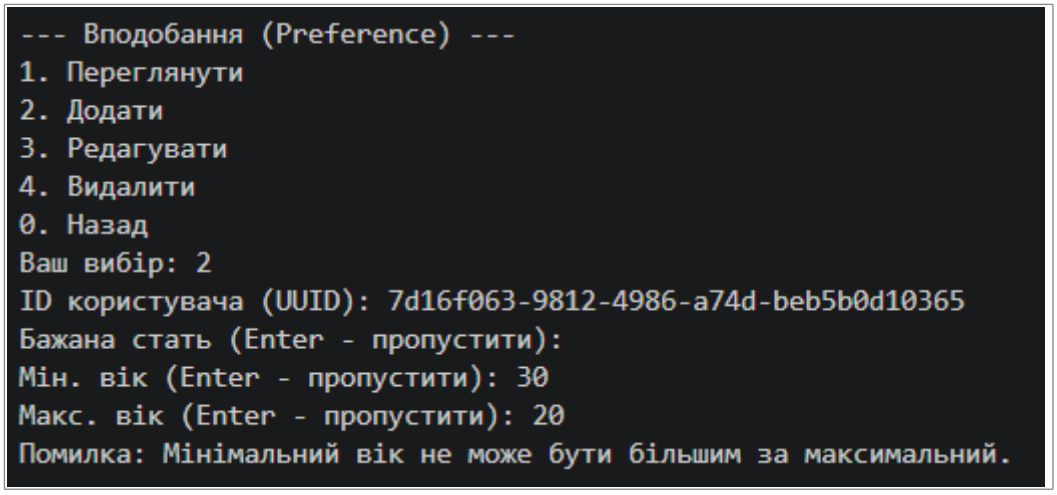
Неправильний UUID



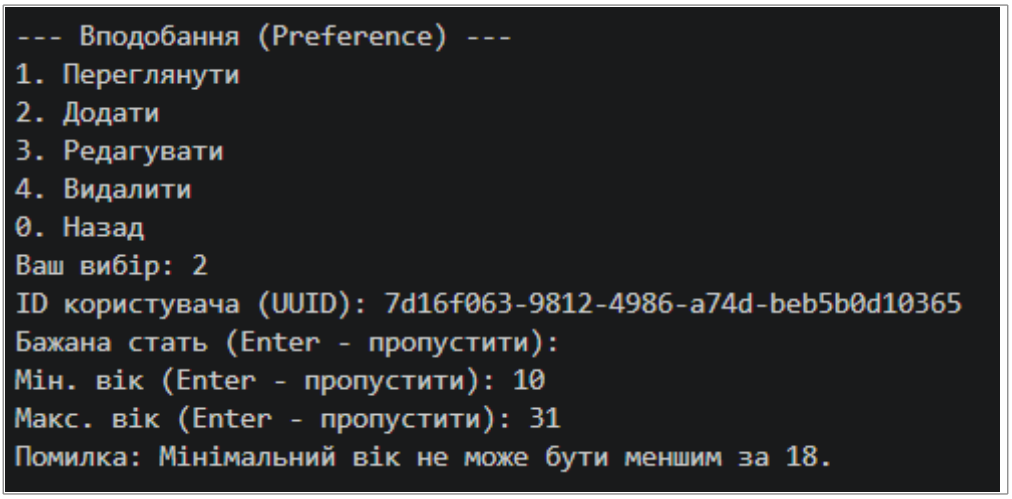
Другий профіль для того самого користувача



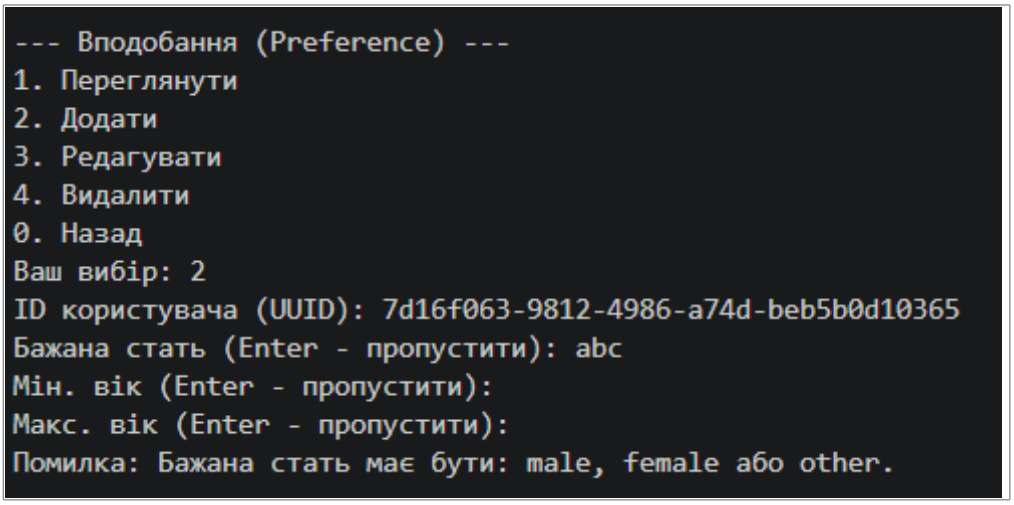
Профіль для користувача, якого немає (порушення FOREIGN KEY)



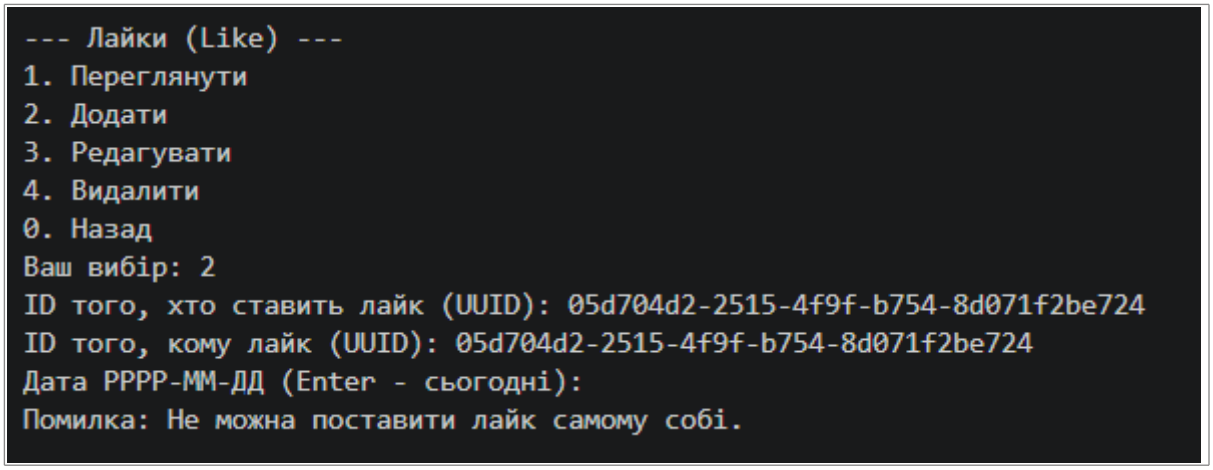
Мінімальний вік більший за максимальний



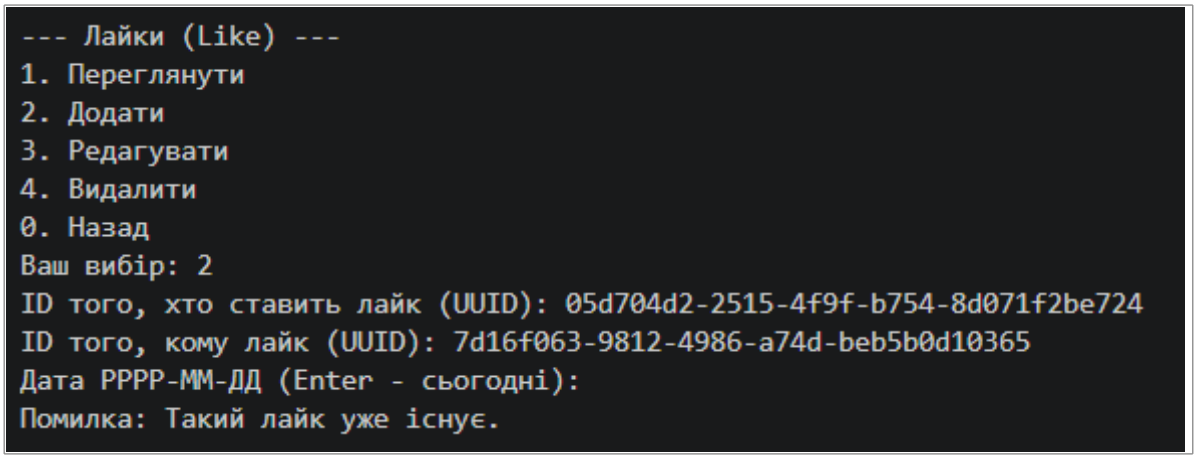
Мінімальний вік менший за 18



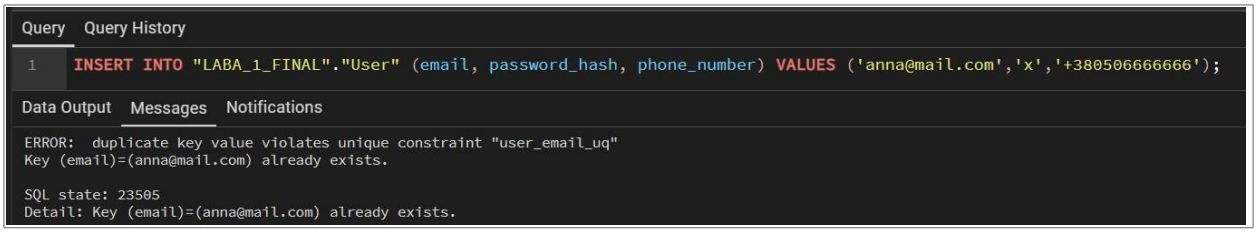
Недопустима бажана стать



Лайк самому собі



Повторний лайк



Для порівняння: той самий INSERT прямо в pgAdmin. Тут видно системну помилку PostgreSQL, а програма її користувачу не показує

Зовнішні ключі та видалення

У схемі всі зовнішні ключі створені з ON DELETE CASCADE. Тому коли видаляється користувач, разом з ним видаляються його профіль, налаштування і всі лайки (і поставлені, і отримані). Щоб користувач бачив, що буде видалено, програма перед видаленням рахує пов'язані записи, показує їх і питає підтвердження.

```
def delete_flow(conn, cfg):
    row_id = parse("uuid", v.ask("ID запису: "))
    deps = crud.count_dependents(conn, cfg, row_id)
    total = sum(n for _, n in deps)
    if total:
        v.message("Із цим записом пов'язані (зовнішні ключі):")
        for label, n in deps:
            v.message(f"  - {label}: {n}")
        if v.ask("Підтвердити вилучення? (так/ні): ").lower() not in _YES:
            v.message("Вилучення скасовано.")
            return
    affected = crud.delete_row(conn, cfg, row_id)
    if not affected:
        v.message("Запис з таким id не знайдено.")
    elif total:
        v.message(f"Видалено. Разом із ним каскадно вилучено "
                  f"пов'язаних записів: {total}.")
    else:
        v.message("Видалено.")
```

Якщо в запису немає залежних записів, програма підтвердження не питає і просто видаляє його.

```
--- Користувачі (User) ---
1. Переглянути
2. Додати
3. Редагувати
4. Видалити
0. Назад
Ваш вибір: 4
ID запису: 4f105dda-5e85-4b8d-93dd-3027d0292f35
Видалено.
```

Видалення Petro, у якого немає залежних записів: підтвердження не питає, «Видалено.»

Query | Query History

```
1  SELECT * FROM "LABA_1_FINAL"."User";
```

Data Output | Messages | Notifications

Showing rows: 1 to 2

	user_id [PK] uuid	email character varying (255)	password_hash character varying (255)	phone_number character varying (30)	created_at date	is_verified boolean
1	8d4b8288-66e9-4131-8859-1331e4e4f...	ivan@mail.com	hash_ivan	+380502222222	2026-11-08	false
2	a756a035-faf2-4f46-a629-797937ead8...	anna@mail.com	hash_anna	+380503333444	2007-03-29	true

Таблиця User у pgAdmin після видалення: лишились Ivan і Anna

Для прикладу далі використовується Anna. У неї є профіль, налаштування, один лайк, який вона поставила (Anna - Ivan), і один, який вона отримала (Ivan - Anna). Це чотири залежні записи. Стан таблиць перед видаленням показаний вище, у розділі «Додавання і перегляд»: у кожній таблиці по два записи.

Для перевірки ще однієї дії зробив скрипт sql/demo_restrict_on.sql. Він змінює зовнішні ключі таблиці Like на ON DELETE RESTRICT, а sql/demo_restrict_off.sql повертає CASCADE. Після першого скрипта ключі like_from_fk і like_to_fk мають тип r (RESTRICT), а ключі профілів і налаштувань залишаються c (CASCADE).

Query | Query History

```
1  SET search_path TO "LABA_1_FINAL";
2
3  BEGIN;
4  ALTER TABLE "Like" DROP CONSTRAINT like_from_fk;
5  ALTER TABLE "Like" DROP CONSTRAINT like_to_fk;
6  ALTER TABLE "Like"
7      ADD CONSTRAINT like_from_fk FOREIGN KEY (user_id_from)
8          REFERENCES "User" (user_id) ON UPDATE NO ACTION ON DELETE RESTRICT;
9  ALTER TABLE "Like"
10     ADD CONSTRAINT like_to_fk FOREIGN KEY (user_id_to)
11         REFERENCES "User" (user_id) ON UPDATE NO ACTION ON DELETE RESTRICT;
12 COMMIT;
13
14 -- Перевірка: confdeltype = 'r' (restrict) для like_from_fk та like_to_fk
15 SELECT conname, confdeltype FROM pg_constraint
16 WHERE connamespace = '"LABA_1_FINAL"'::regnamespace AND contype = 'f'
17 ORDER BY conname;
```

Data Output | Messages | Notifications

SQL

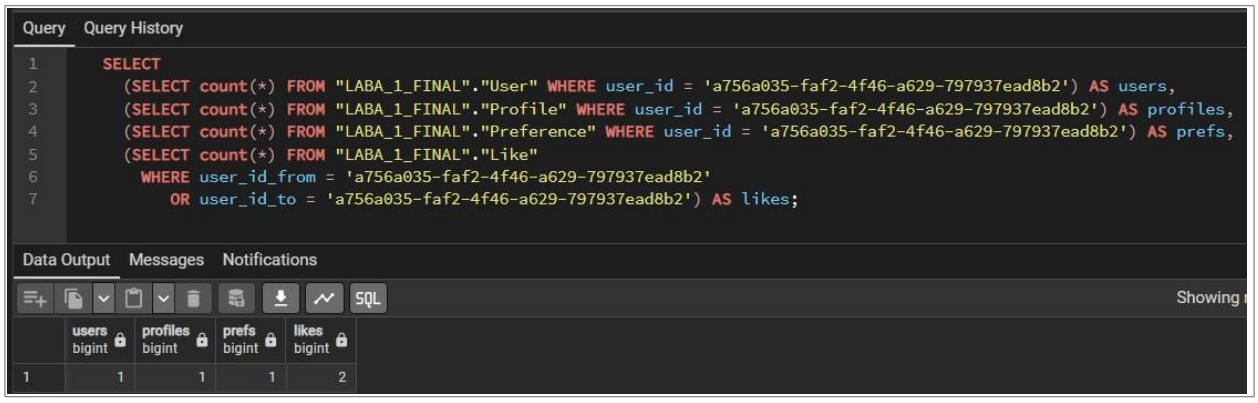
	conname name	confdeltype "char"
1	like_from_fk	r
2	like_to_fk	r
3	preference_user_fk	c
4	profile_user_fk	c

Зміна ключів Like на RESTRICT і перевірка confdeltype (r - RESTRICT, c - CASCADE)

При RESTRICT видалити Anna не вийде: у неї є лайки, на які посилаються ключі. DELETE виконується цілком або не виконується взагалі, тому профіль і налаштування Anna теж лишаються на місці, а програма пише «Не можна видалити: на цей запис посилаються інші записи.». Для RESTRICT PostgreSQL повертає помилку з кодом 23001 (restrict_violation), а не 23503, як для звичайного порушення зовнішнього ключа, тому в errors.py обробляються обидва типи помилок.

```
--- Користувачі (User) ---
1. Переглянути
2. Додати
3. Редагувати
4. Видалити
0. Назад
Ваш вибір: 4
ID запису: a756a035-faf2-4f46-a629-797937ead8b2
Із цим записом пов'язані (зовнішні ключі):
  - профілі: 1
  - налаштування: 1
  - лайки, поставлені користувачем: 1
  - лайки, отримані користувачем: 1
Підтвердити вилучення? (так/ні): так
Помилка: Не можна видалити: на цей запис посилаються інші записи.
```

Спроба видалити Anna при RESTRICT: програма показує залежні записи, після підтвердження пише, що видалити не можна



Перевірка в pgAdmin після відмови: Anna, її профіль, налаштування і два лайки на місці (1 / 1 / 1 / 2)

Query Query History

```sql
SET search_path TO "LABA_1_FINAL";

BEGIN;
ALTER TABLE "Like" DROP CONSTRAINT like_from_fk;
ALTER TABLE "Like" DROP CONSTRAINT like_to_fk;
ALTER TABLE "Like"
    ADD CONSTRAINT like_from_fk FOREIGN KEY (user_id_from)
        REFERENCES "User" (user_id) ON UPDATE NO ACTION ON DELETE CASCADE;
ALTER TABLE "Like"
    ADD CONSTRAINT like_to_fk FOREIGN KEY (user_id_to)
        REFERENCES "User" (user_id) ON UPDATE NO ACTION ON DELETE CASCADE;
COMMIT;

-- Перевірка: confdeltype = 'c' (cascade) для всіх чотирьох зовнішніх ключів
SELECT conname, confdeltype FROM pg_constraint
WHERE connamespace = '"LABA_1_FINAL"'::regnamespace AND contype = 'f'
ORDER BY conname;
```

Data Output Messages Notifications

	conname name	confdeltype "char"
1	like_from_fk	c
2	like_to_fk	c
3	preference_user_fk	c
4	profile_user_fk	c

Повернення CASCADE (sql/demo_restrict_off.sql): confdeltype = c для всіх чотирьох ключів

Після повернення CASCADE видалення Anna проходить: програма показує чотири залежні записи, після підтвердження видаляє користувача і всі ці записи.

```
--- Користувачі (User) ---
1. Переглянути
2. Додати
3. Редагувати
4. Видалити
0. Назад
Ваш вибір: 4
ID запису: a756a035-faf2-4f46-a629-797937ead8b2
Із цим записом пов'язані (зовнішні ключі):
  - профілі: 1
  - налаштування: 1
  - лайки, поставлені користувачем: 1
  - лайки, отримані користувачем: 1
Підтвердити вилучення? (так/ні): так
Видалено. Разом із ним каскадно вилучено пов'язаних записів: 4.
```

Видалення Anna при CASCADE: програма показує залежні записи і видаляє їх разом з користувачем

Query Query History

```sql
SELECT * FROM "LABA_1_FINAL"."User";
```

Data Output Messages Notifications

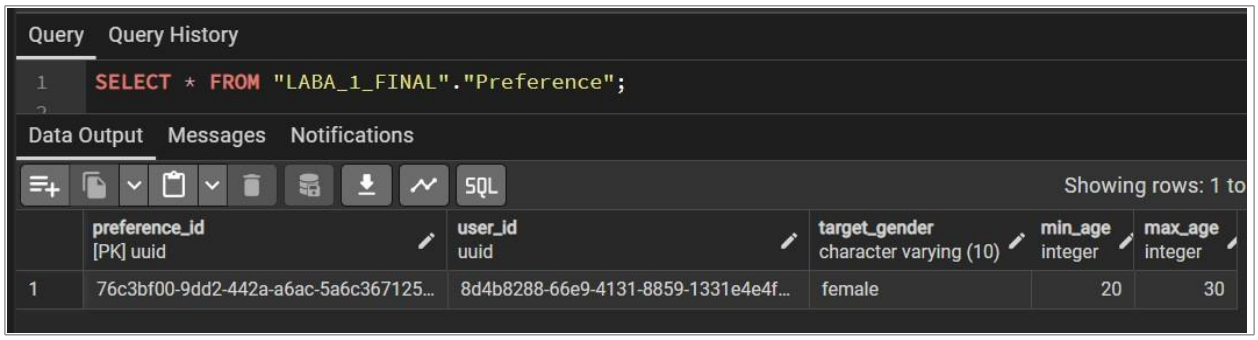
Showing rows: 1 to 1

	user_id [PK] uuid	email character varying (255)	password_hash character varying (255)	phone_number character varying (30)	created_at date	is_verified boolean
1	8d4b8288-66e9-4131-8859-1331e4e4f...	ivan@mail.com	hash_ivan	+380502222222	2026-11-08	false

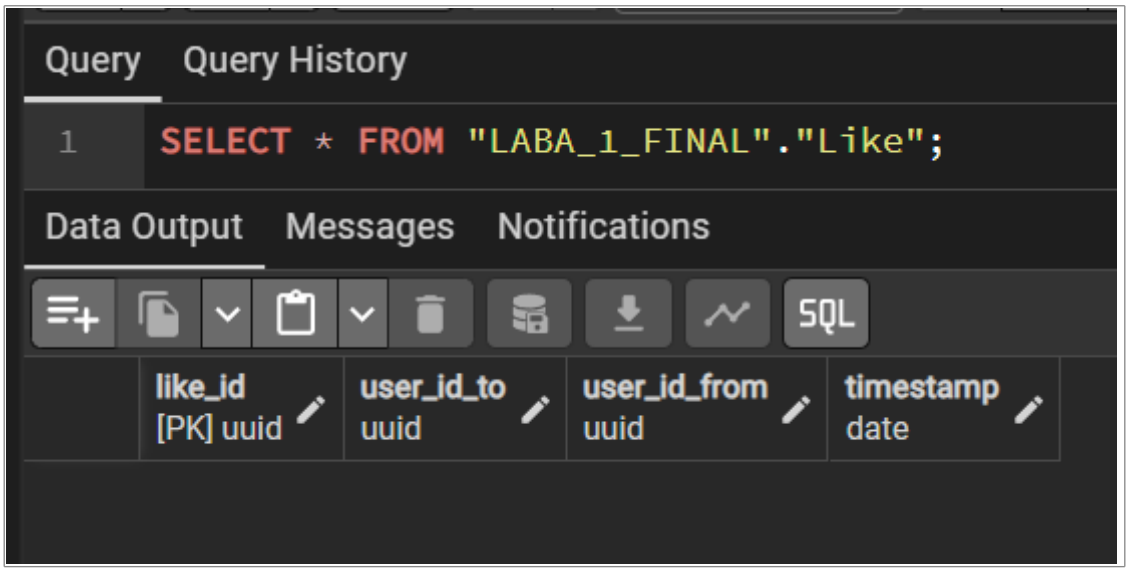
Після видалення в User залишився тільки Ivan



У Profile залишився тільки профіль Ivan



У Preference залишилися тільки вподобання Ivan



Таблиця Like порожня, бо обидва лайки Anna видалились разом з нею

Пункт №2. Генерація даних

Як це зроблено

Дані генеруються тільки SQL-запитами (файл model/generator.py), у Python значення не створюються. Використані generate_series, random, floor, md5, lpad, масиви ARRAY[...] і додавання до дат. Кількість записів вводить користувач, вона передається в запит параметром (%s::int). Програма не приймає число менше 1 і більше 1 000 000. Пункт «Повний набір» створює користувачів, профілі, налаштування і лайки в одній транзакції.

SQL-запити

У запитах %s - це кількість записів, яку ввів користувач.

Користувачі

```
WITH base AS (
    SELECT coalesce(max(substr(phone_number, 7)::int), 0) AS s
    FROM "User"
    WHERE phone_number ~ '^\+38099[0-9]{7}$'
)
INSERT INTO "User" (email, password_hash, phone_number, created_at, is_verified)
SELECT 'user' || (base.s + g.i) || '@example.com',
       md5(random()::text),
       '+38099' || lpad((base.s + g.i)::text, 7, '0'),
       DATE '2023-01-01' + floor(random() * 1300)::int,
       random() < 0.3
FROM generate_series(1, %s::int) AS g(i)
CROSS JOIN base
```

Профілі (тільки для користувачів без профілю)

```
WITH tgt AS MATERIALIZED (
    SELECT u.user_id, row_number() OVER () AS rn
    FROM "User" u
    WHERE NOT EXISTS (SELECT 1 FROM "Profile" p WHERE p.user_id = u.user_id)
    LIMIT %s::int
), rnd AS MATERIALIZED (
    SELECT g.i AS rn, random() AS r_name, random() AS r_birth, random() AS r_gender
    FROM generate_series(1, (SELECT count(*)::int FROM tgt)) AS g(i)
)
INSERT INTO "Profile" (user_id, first_name, birth_date, gender)
SELECT t.user_id,
       (ARRAY['Anna','Ivan','Olena','Petro','Maria',
              'Andrii','Sofiia','Taras','Iryna','Oleh'])[1 + floor(r.r_name * 10)::int],
       DATE '1960-01-01' + floor(r.r_birth * 17000)::int,
       (ARRAY['male','female','other'])[1 + floor(r.r_gender * 3)::int]
FROM tgt t JOIN rnd r ON r.rn = t.rn
```

Налаштування (тільки для користувачів без налаштувань)

```
WITH tgt AS MATERIALIZED (
    SELECT u.user_id, row_number() OVER () AS rn
    FROM "User" u
    WHERE NOT EXISTS (SELECT 1 FROM "Preference" p WHERE p.user_id = u.user_id)
    LIMIT %s::int
), rnd AS MATERIALIZED (
    SELECT g.i AS rn, random() AS r_gender, random() AS r_min, random() AS r_span
    FROM generate_series(1, (SELECT count(*)::int FROM tgt)) AS g(i)
)
INSERT INTO "Preference" (user_id, target_gender, min_age, max_age)
SELECT t.user_id,
       (ARRAY['male','female','other'])[1 + floor(r.r_gender * 3)::int],
       18 + floor(r.r_min * 20)::int,
       18 + floor(r.r_min * 20)::int + floor(r.r_span * 40)::int
FROM tgt t JOIN rnd r ON r.rn = t.rn
```

Лайки (випадкові пари користувачів)

```
WITH u AS MATERIALIZED (
    SELECT user_id, row_number() OVER (ORDER BY user_id) AS rn FROM "User"
), cnt AS (
    SELECT count(*)::int AS c FROM u
), pairs AS MATERIALIZED (
    SELECT 1 + floor(random() * cnt.c)::int AS f,
           1 + floor(random() * cnt.c)::int AS t,
           DATE '2024-01-01' + floor(random() * 900)::int AS d
    FROM generate_series(1, %s::int) AS g(i) CROSS JOIN cnt
)
INSERT INTO "Like" (user_id_from, user_id_to, "timestamp")
SELECT uf.user_id, ut.user_id, p.d
FROM pairs p
JOIN u uf ON uf.rn = p.f
JOIN u ut ON ut.rn = p.t
WHERE uf.user_id <> ut.user_id
ON CONFLICT DO NOTHING
```

Як дотримуються обмеження

Обмеження	**Як зроблено**
FOREIGN KEY	Профілі й налаштування створюються для user_id, які беруться з таблиці User. Для лайків випадкові номери беруться в межах кількості користувачів і з'єднуються з нумерованим списком User.
NOT NULL	Кожен INSERT заповнює всі обов'язкові поля: email, хеш (md5(random()::text)), телефон, дати, ім'я, стать. Для is_verified береться random() < 0.3.
UNIQUE для email і телефону	Номер рахується від найбільшого вже використаного суфікса телефону плюс і. Тому при повторній генерації нумерація продовжується (user11@example.com і т.д.).
UNIQUE для Profile і Preference	Беруться тільки користувачі, у яких ще немає запису (NOT EXISTS). Якщо таких менше, ніж просили, додається стільки, скільки є.
UNIQUE для лайків і CHECK на сам себе	Умова uf.user_id <> ut.user_id і ON CONFLICT DO NOTHING. Пари, які не підходять, пропускаються, тому лайків може бути трохи менше за N.
CHECK	Стать береться з масиву допустимих значень. Дата народження від 1960 року. Мінімальний вік 18-37, максимальний дорівнює мінімальному плюс від 0 до 39, тому min не більший за max і max менший за 120. Email має вигляд userN@example.com, це підходить під шаблон %_@_%.__%.

Невелика кількість даних

Query Query History

```
1  TRUNCATE TABLE "LABA_1_FINAL"."Like", "LABA_1_FINAL"."Preference",
2                 "LABA_1_FINAL"."Profile", "LABA_1_FINAL"."User";
```

Data Output Messages Notifications

```
TRUNCATE TABLE

Query returned successfully in 91 msec.
```

Очищення таблиць

Query Query History

```
1  SELECT 'User' AS table_name, count(*) AS rows_count FROM "User"
2  UNION ALL SELECT 'Profile', count(*) FROM "Profile"
3  UNION ALL SELECT 'Preference', count(*) FROM "Preference"
4  UNION ALL SELECT 'Like', count(*) FROM "Like";
```

Data Output Messages Notifications

	table_name text	rows_count bigint
1	User	0
2	Profile	0
3	Preference	0
4	Like	0

Усі таблиці порожні

```
===== РГР: ОБЕРІТЬ ПУНКТ ЗАВДАННЯ =====
1. Пункт 1 - перегляд, внесення, редагування, вилучення даних
2. Пункт 2 - генерація даних (SQL)
3. Пункт 3 - пошук (параметризовані запити)
0. Вихід
Ваш вибір: 2

--- Генерація даних (SQL) ---
1. Користувачі
2. Профілі (для користувачів без профілю)
3. Налаштування (для користувачів без налаштувань)
4. Лайки (випадкові пари користувачів)
5. Повний набір (усі чотири таблиці)
0. Назад
Ваш вибір: 1
Кількість записів: 10
Додано записів: 10. Час виконання: 28.3 мс
```

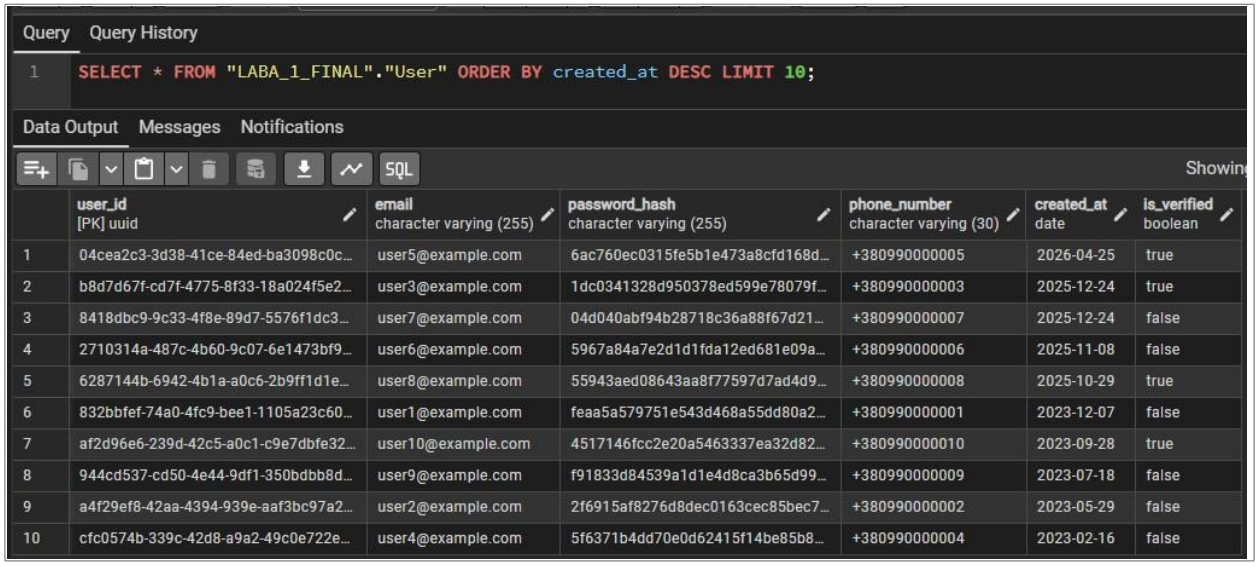
Генерація користувачів: кількість (10) вводиться з клавіатури, виводиться час у мс

```
--- Генерація даних (SQL) ---
1. Користувачі
2. Профілі (для користувачів без профілю)
3. Налаштування (для користувачів без налаштувань)
4. Лайки (випадкові пари користувачів)
5. Повний набір (усі чотири таблиці)
0. Назад
Ваш вибір: 2
Кількість записів: 10
Додано записів: 10. Час виконання: 5.7 мс

--- Генерація даних (SQL) ---
1. Користувачі
2. Профілі (для користувачів без профілю)
3. Налаштування (для користувачів без налаштувань)
4. Лайки (випадкові пари користувачів)
5. Повний набір (усі чотири таблиці)
0. Назад
Ваш вибір: 3
Кількість записів: 10
Додано записів: 10. Час виконання: 22.8 мс

--- Генерація даних (SQL) ---
1. Користувачі
2. Профілі (для користувачів без профілю)
3. Налаштування (для користувачів без налаштувань)
4. Лайки (випадкові пари користувачів)
5. Повний набір (усі чотири таблиці)
0. Назад
Ваш вибір: 4
Кількість записів: 10
Додано записів: 9. Час виконання: 4.1 мс
```

Генерація профілів, налаштувань і лайків. Лайків додалось 9, бо одна пара не підійшла під обмеження



Query | Query History

```
1  SELECT * FROM "LABA_1_FINAL"."User" ORDER BY created_at DESC LIMIT 10;
```

Data Output | Messages | Notifications

	user_id [PK] uuid	email character varying (255)	password_hash character varying (255)	phone_number character varying (30)	created_at date	is_verified boolean
1	04cea2c3-3d38-41ce-84ed-ba3098c0c...	user5@example.com	6ac760ec0315fe5b1e473a8cfd168d...	+380990000005	2026-04-25	true
2	b8d7d67f-cd7f-4775-8f33-18a024f5e2...	user3@example.com	1dc0341328d950378ed599e78079f...	+380990000003	2025-12-24	true
3	8418dbc9-9c33-4f8e-89d7-5576f1dc3...	user7@example.com	04d040abf94b28718c36a88f67d21...	+380990000007	2025-12-24	false
4	2710314a-487c-4b60-9c07-6e1473bf9...	user6@example.com	5967a84a7e2d1d1fda12ed681e09a...	+380990000006	2025-11-08	false
5	6287144b-6942-4b1a-a0c6-2b9ff1d1e...	user8@example.com	55943aed08643aa8f77597d7ad4d9...	+380990000008	2025-10-29	true
6	832bbfef-74a0-4fc9-bee1-1105a23c60...	user1@example.com	feaa5a579751e543d468a55dd80a2...	+380990000001	2023-12-07	false
7	af2d96e6-239d-42c5-a0c1-c9e7dbfe32...	user10@example.com	4517146fcc2e20a5463337ea32d82...	+380990000010	2023-09-28	true
8	944cd537-cd50-4e44-9df1-350bdbb8d...	user9@example.com	f91833d84539a1d1e4d8ca3b65d99...	+380990000009	2023-07-18	false
9	a4f29ef8-42aa-4394-939e-aaf3bc97a2...	user2@example.com	2f6915af8276d8dec0163cec85bec7...	+380990000002	2023-05-29	false
10	cfc0574b-339c-42d8-a9a2-49c0e722e...	user4@example.com	5f6371b4dd70e0d62415f14be85b8...	+380990000004	2023-02-16	false

Таблиця User після генерації

Query Query History

```
1  SELECT * FROM "LABA_1_FINAL"."Profile" LIMIT 10;
```

Data Output Messages Notifications

	profile_id [PK] uuid	user_id uuid	first_name character varying (100)	birth_date date	gender character varying (10)
1	862e8755-f5f4-4357-ba66-5f3dd74a1...	832bbfef-74a0-4fc9-bee1-1105a23c60...	Ivan	1987-05-17	male
2	be2415b8-b0b7-4125-b7bc-acc9093a2...	a4f29ef8-42aa-4394-939e-aaf3bc97a2...	Andrii	2003-10-11	female
3	b2a37ae2-debd-46fb-a4a9-950ba0722...	b8d7d67f-cd7f-4775-8f33-18a024f5e2...	Ivan	1971-09-13	other
4	8db40ba8-3f6c-47f9-a014-3f9814a3fe...	cfc0574b-339c-42d8-a9a2-49c0e722e...	Ivan	1960-08-18	male
5	3a6dbfa8-c292-4ede-86dc-cfbb18bc5...	04cea2c3-3d38-41ce-84ed-ba3098c0c...	Anna	1962-12-18	female
6	65726036-6388-400f-b5c4-110766994...	2710314a-487c-4b60-9c07-6e1473bf9...	Petro	1964-08-18	male
7	9907160d-8489-41aa-9195-dc6223e9...	8418dbc9-9c33-4f8e-89d7-5576f1dc3...	Oleh	1976-06-04	male
8	bf595e98-0d82-44e3-b17e-aa73cc1b5...	6287144b-6942-4b1a-a0c6-2b9ff1d1e...	Petro	1966-09-27	other
9	65c10f89-eed3-41a6-b77d-6e7c338be...	944cd537-cd50-4e44-9df1-350bdbb8d...	Taras	1995-03-10	female
10	6363b394-d97c-41b0-b894-8f52dec51...	af2d96e6-239d-42c5-a0c1-c9e7dbfe32...	Iryna	1984-02-01	female

Таблиця Profile після генерації

Query Query History

```
1  SELECT * FROM "LABA_1_FINAL"."Preference" LIMIT 10;
```

Data Output Messages Notifications

	preference_id [PK] uuid	user_id uuid	target_gender character varying (10)	min_age integer	max_age integer
1	b675d8ed-bc28-4ab2-8dc8-35e3b4e8...	832bbfef-74a0-4fc9-bee1-1105a23c60...	male	37	37
2	862e624a-a2c4-4d03-940b-deedefc32...	a4f29ef8-42aa-4394-939e-aaf3bc97a2...	male	35	36
3	a1c8d2b0-00d9-4bb4-8758-2f9ddf262...	b8d7d67f-cd7f-4775-8f33-18a024f5e2...	female	24	43
4	805b4379-eccf-4854-b4bb-a1310baab...	cfc0574b-339c-42d8-a9a2-49c0e722e...	male	25	34
5	9893ebef-21af-4b13-968e-99342e9b3...	04cea2c3-3d38-41ce-84ed-ba3098c0c...	male	19	38
6	11fa42dc-f0c1-48c5-9ad4-a6dd7603e...	2710314a-487c-4b60-9c07-6e1473bf9...	other	29	45
7	e7eed2b0-be3a-4611-8bbf-cef8d378d...	8418dbc9-9c33-4f8e-89d7-5576f1dc3...	male	32	47
8	392d8c0d-5e83-4743-92bc-37959561...	6287144b-6942-4b1a-a0c6-2b9ff1d1e...	female	36	66
9	0654b99c-2163-4cbf-8677-fab539ec6...	944cd537-cd50-4e44-9df1-350bdbb8d...	female	22	30
10	e94e0b75-f895-4dbd-9bc5-5af1869ae...	af2d96e6-239d-42c5-a0c1-c9e7dbfe32...	male	36	50

Таблиця Preference після генерації

Query Query History

```
1  SELECT * FROM "LABA_1_FINAL"."Like" LIMIT 10;
```

Data Output Messages Notifications

	like_id [PK] uuid	user_id_to uuid	user_id_from uuid	timestamp date
1	720345ed-148b-4b0f-99cd-78d642955...	04cea2c3-3d38-41ce-84ed-ba3098c0...	cfc0574b-339c-42d8-a9a2-49c0e722e...	2024-02-20
2	2c2b543d-a7a2-4ab2-85e5-37044e11a...	8418dbc9-9c33-4f8e-89d7-5576f1dc3...	af2d96e6-239d-42c5-a0c1-c9e7dbfe32...	2024-08-16
3	b856a254-7397-4e3b-a178-2ff748b33a...	a4f29ef8-42aa-4394-939e-aaf3bc97a...	8418dbc9-9c33-4f8e-89d7-5576f1dc3...	2024-11-03
4	f8d2635e-b89f-460b-87d0-0ccdef83ce8b	a4f29ef8-42aa-4394-939e-aaf3bc97a...	2710314a-487c-4b60-9c07-6e1473bf9...	2025-09-18
5	d526489c-2609-4dbb-9699-7bb2478e8...	af2d96e6-239d-42c5-a0c1-c9e7dbfe3...	cfc0574b-339c-42d8-a9a2-49c0e722e...	2024-08-08
6	d7f79fc1-45ab-4c83-b623-c24ab090e4...	af2d96e6-239d-42c5-a0c1-c9e7dbfe3...	a4f29ef8-42aa-4394-939e-aaf3bc97a2...	2025-12-10
7	697c471b-e78b-4a01-834f-fffe13281b8c	af2d96e6-239d-42c5-a0c1-c9e7dbfe3...	6287144b-6942-4b1a-a0c6-2b9ff1d1e...	2024-08-07
8	385cb26c-8b25-4633-87fe-c8e489485d...	b8d7d67f-cd7f-4775-8f33-18a024f5e2...	8418dbc9-9c33-4f8e-89d7-5576f1dc3...	2025-07-13
9	81dae8f7-0225-4d74-ac05-a7756c6512...	b8d7d67f-cd7f-4775-8f33-18a024f5e2...	2710314a-487c-4b60-9c07-6e1473bf9...	2025-10-11

Таблиця Like після генерації

```
--- Генерація даних (SQL) ---
1. Користувачі
2. Профілі (для користувачів без профілю)
3. Налаштування (для користувачів без налаштувань)
4. Лайки (випадкові пари користувачів)
5. Повний набір (усі чотири таблиці)
0. Назад
Ваш вибір: 1
Кількість записів: 5
Додано записів: 5. Час виконання: 2.1 мс
```

Повторна генерація: ще 5 користувачів

Query Query History

```
1  SELECT count(*), count(DISTINCT email), count(DISTINCT phone_number) FROM "LABA_1_FINAL"."User";
```

Data Output Messages Notifications

	count bigint	count bigint	count bigint
1	15	15	15

Тепер 15 записів, 15 різних email і 15 різних телефонів, тобто UNIQUE не порушено

```
--- Генерація даних (SQL) ---
1. Користувачі
2. Профілі (для користувачів без профілю)
3. Налаштування (для користувачів без налаштувань)
4. Лайки (випадкові пари користувачів)
5. Повний набір (усі чотири таблиці)
0. Назад
Ваш вибір: 2
Кількість записів: 10
Додано записів: 5. Час виконання: 2.1 мс
```

Генерація профілів: просили 10, додано 5, бо профілю не мали тільки 5 користувачів

```
--- Генерація даних (SQL) ---
1. Користувачі
2. Профілі (для користувачів без профілю)
3. Налаштування (для користувачів без налаштувань)
4. Лайки (випадкові пари користувачів)
5. Повний набір (усі чотири таблиці)
0. Назад
Ваш вибір: 2
Кількість записів: -5
Помилка введення: число має бути від 1 до 1000000

--- Генерація даних (SQL) ---
1. Користувачі
2. Профілі (для користувачів без профілю)
3. Налаштування (для користувачів без налаштувань)
4. Лайки (випадкові пари користувачів)
5. Повний набір (усі чотири таблиці)
0. Назад
Ваш вибір: 2
Кількість записів: 0
Помилка введення: число має бути від 1 до 1000000
```

Числа -5 і 0 програма не приймає

100 000 записів

Query | Query History

```
1  TRUNCATE TABLE "LABA_1_FINAL"."Like", "LABA_1_FINAL"."Preference",
2                 "LABA_1_FINAL"."Profile", "LABA_1_FINAL"."User";
```

Data Output | Messages | Notifications

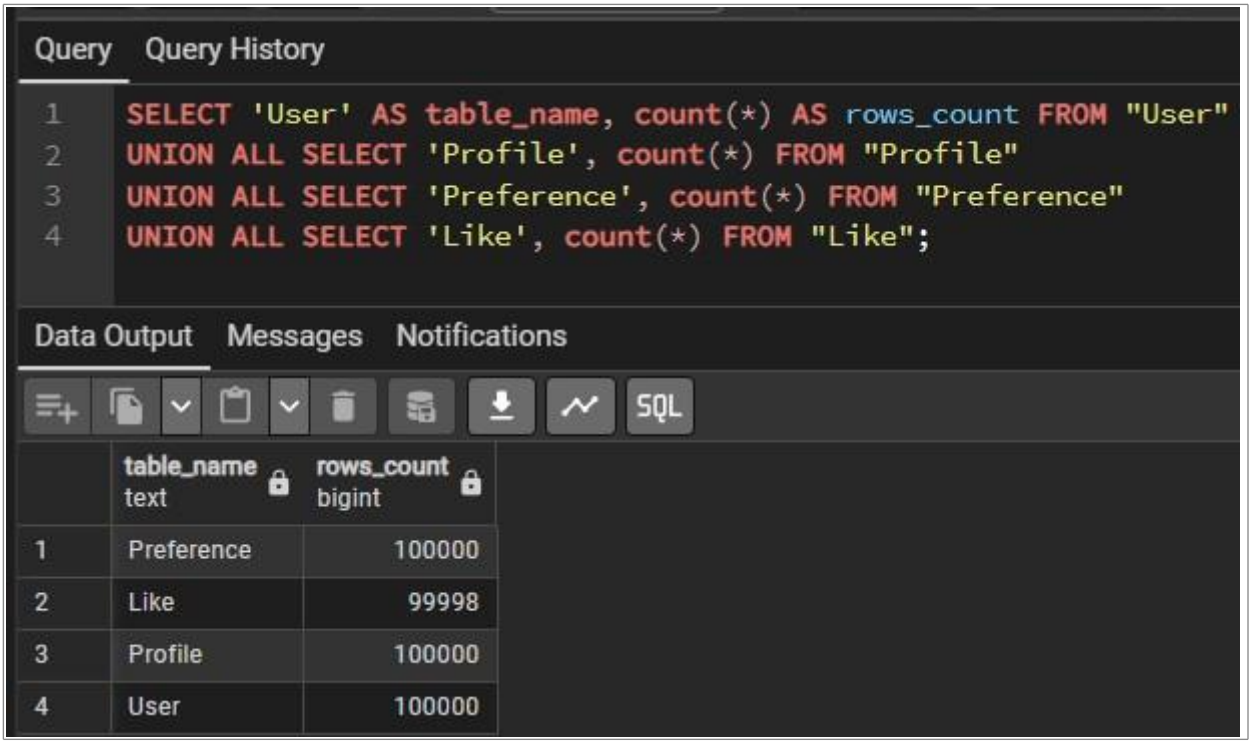
```
TRUNCATE TABLE

Query returned successfully in 96 msec.
```

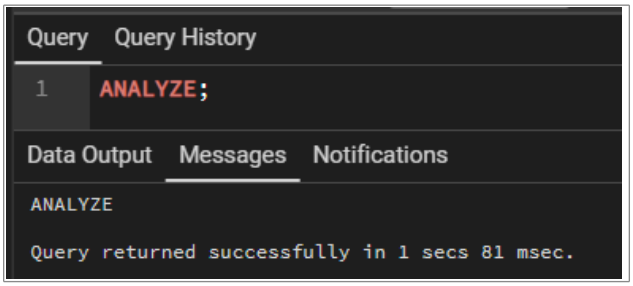
Очищення таблиць перед великою генерацією

```
--- Генерація даних (SQL) ---
1. Користувачі
2. Профілі (для користувачів без профілю)
3. Налаштування (для користувачів без налаштувань)
4. Лайки (випадкові пари користувачів)
5. Повний набір (усі чотири таблиці)
0. Назад
Ваш вибір: 5
Кількість користувачів (і лайків): 100000
Користувачі: додано 100000 записів за 3528.0 мс
Профілі: додано 100000 записів за 3403.8 мс
Налаштування: додано 100000 записів за 2826.8 мс
Лайки: додано 99998 записів за 4752.9 мс
```

Повний набір: по 100 000 записів у User, Profile, Preference і 99 998 у Like

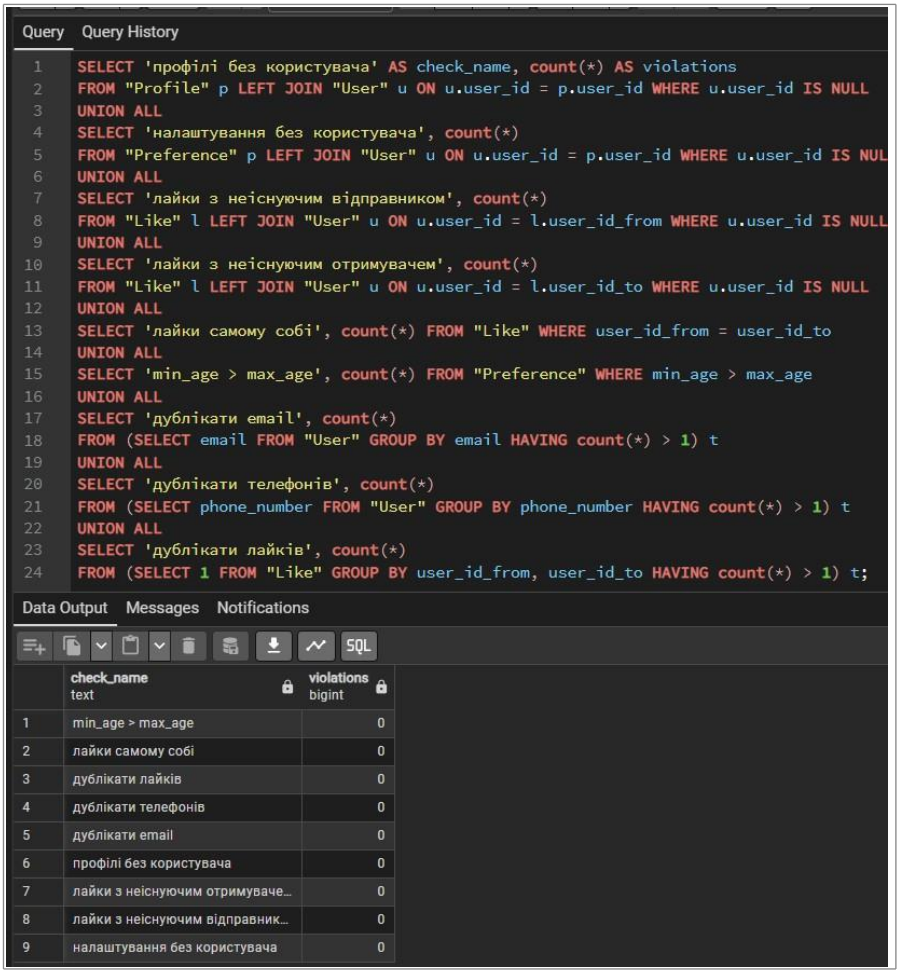


Кількість записів у таблицях



Оновлення статистики командою ANALYZE

Таблиця	**Додано записів**	**Час, мс**
User	100 000	3528.0
Profile	100 000	3403.8
Preference	100 000	2826.8
Like	99 998 (2 пари не підійшли під обмеження)	4752.9

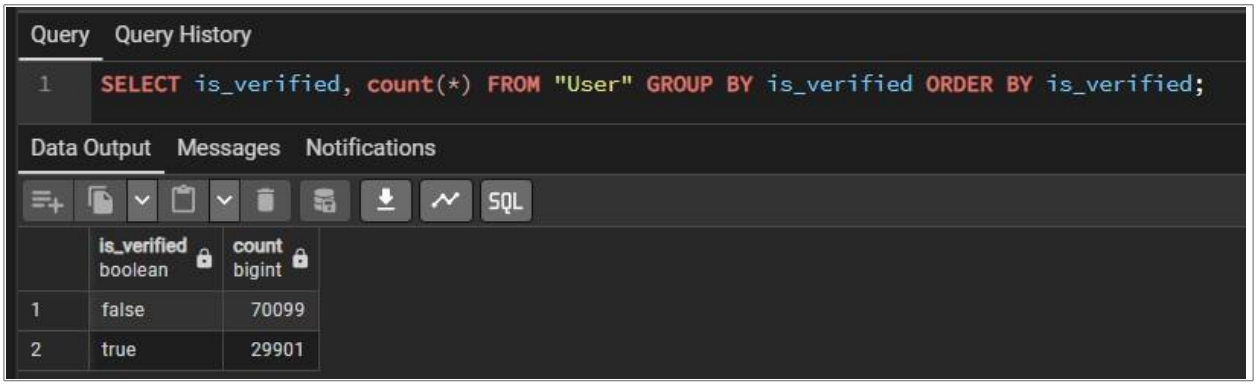


```
Query  Query History
1  SELECT 'профілі без користувача' AS check_name, count(*) AS violations
2  FROM "Profile" p LEFT JOIN "User" u ON u.user_id = p.user_id WHERE u.user_id IS NULL
3  UNION ALL
4  SELECT 'налаштування без користувача', count(*)
5  FROM "Preference" p LEFT JOIN "User" u ON u.user_id = p.user_id WHERE u.user_id IS NUL
6  UNION ALL
7  SELECT 'лайки з неіснуючим відправником', count(*)
8  FROM "Like" l LEFT JOIN "User" u ON u.user_id = l.user_id_from WHERE u.user_id IS NULL
9  UNION ALL
10 SELECT 'лайки з неіснуючим отримувачем', count(*)
11 FROM "Like" l LEFT JOIN "User" u ON u.user_id = l.user_id_to WHERE u.user_id IS NULL
12 UNION ALL
13 SELECT 'лайки самому собі', count(*) FROM "Like" WHERE user_id_from = user_id_to
14 UNION ALL
15 SELECT 'min_age > max_age', count(*) FROM "Preference" WHERE min_age > max_age
16 UNION ALL
17 SELECT 'дублікати email', count(*)
18 FROM (SELECT email FROM "User" GROUP BY email HAVING count(*) > 1) t
19 UNION ALL
20 SELECT 'дублікати телефонів', count(*)
21 FROM (SELECT phone_number FROM "User" GROUP BY phone_number HAVING count(*) > 1) t
22 UNION ALL
23 SELECT 'дублікати лайків', count(*)
24 FROM (SELECT 1 FROM "Like" GROUP BY user_id_from, user_id_to HAVING count(*) > 1) t;
```

Data Output Messages Notifications

	check_name text	violations bigint
1	min_age > max_age	0
2	лайки самому собі	0
3	дублікати лайків	0
4	дублікати телефонів	0
5	дублікати email	0
6	профілі без користувача	0
7	лайки з неіснуючим отримуваче…	0
8	лайки з неіснуючим відправник…	0
9	налаштування без користувача	0

Перевірка цілісності (sql/check_integrity.sql): у всіх дев'яти перевірках 0 порушень



```
Query  Query History
1  SELECT is_verified, count(*) FROM "User" GROUP BY is_verified ORDER BY is_verified;
```

Data Output Messages Notifications

	is_verified boolean	count bigint
1	false	70099
2	true	29901

Розподіл is_verified: 29 901 true (близько 30 %) і 70 099 false

Пункт №3. Пошук

Які запити зроблені

№	Що шукає	Таблиці	Умови
1	Користувачі за профілем	User, Profile	ILIKE за іменем, стать, діапазон дат народження, true/false для is_verified
2	Найпопулярніші користувачі за лайками	Like, User, Profile	діапазон дат лайка, ILIKE, GROUP BY і HAVING COUNT(*) у діапазоні
3	Підходящі кандидати для користувача	User, Preference, Profile, Like	email, умови з налаштувань (стать, вік), ILIKE, без тих, кого вже лайкнули
4	Користувачі за налаштуваннями	Preference, User, Profile	бажана стать, діапазони min_age і max_age, ILIKE

Усі фільтри необов'язкові. Якщо нічого не вводити (Enter), умова не застосовується. Результат обмежений 50 рядками. Після таблиці програма показує кількість рядків і час виконання в мілісекундах.

Як значення передаються в запит

Тексти запитів лежать у model/search.py з іменованими параметрами, наприклад %(name)s. Значення передаються словником другим аргументом у cur.execute, тому драйвер відправляє їх серверу окремо від SQL, і те, що ввів користувач, не може стати частиною команди. Необов'язкова умова записана так: (%(name)s::text IS NULL OR p.first_name ILIKE %(name)s). Якщо поле порожнє, передається None. Контролер тільки додає % з двох боків, якщо в шаблоні немає % і _, і цей рядок теж іде параметром.

```
def _like(text):
    """Якщо шаблон без % та _, шукаємо входження підрядка."""
    if text is None:
        return None
    return text if ("%" in text or "_" in text) else f"%{text}%"

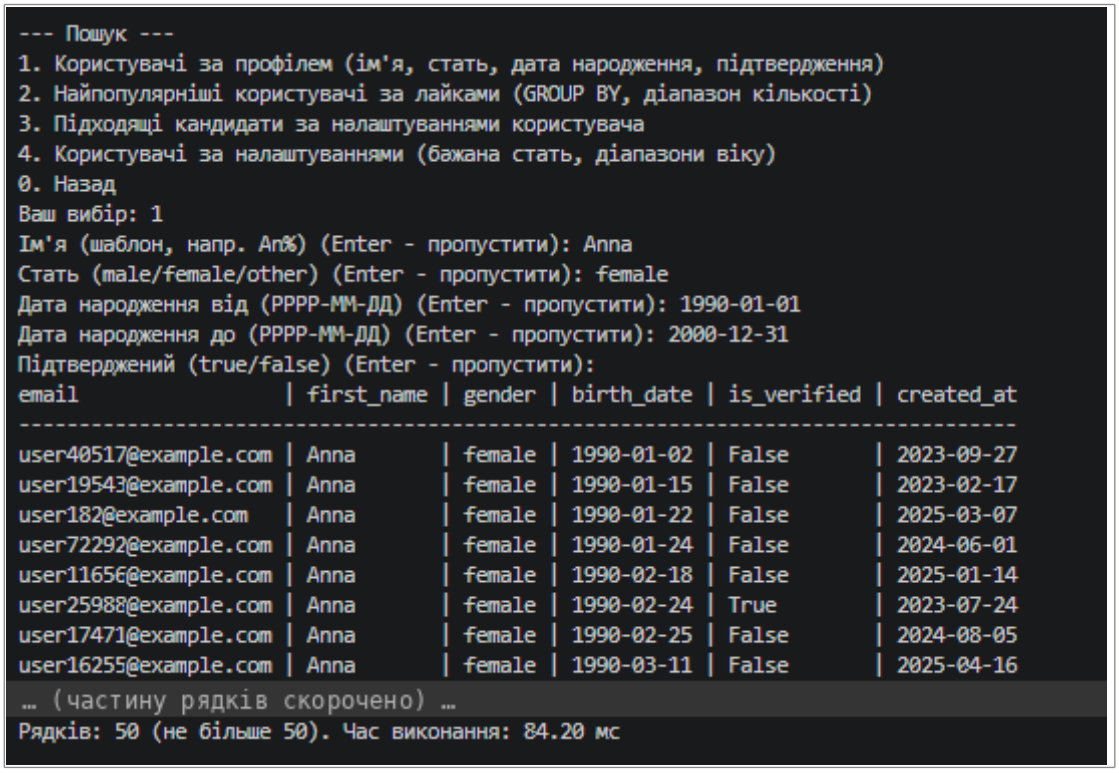
def _run(conn, query, params):
    params["lim"] = LIMIT
    try:
        with conn.cursor() as cur:
            t = time.perf_counter()
            cur.execute(query, params)        # значення передаються окремо від SQL
            rows = cur.fetchall()
            ms = (time.perf_counter() - t) * 1000
            cols = [d.name for d in cur.description]
        conn.commit()
        return cols, rows, ms
    except Exception as e:
        conn.rollback()
        raise DbError(translate(e, "select")) from e


def profiles(conn, name, gender, born_from, born_to, verified):
    return _run(conn, Q_PROFILES, {"name": name, "gender": gender,
                                   "born_from": born_from, "born_to": born_to,
                                   "verified": verified})
```

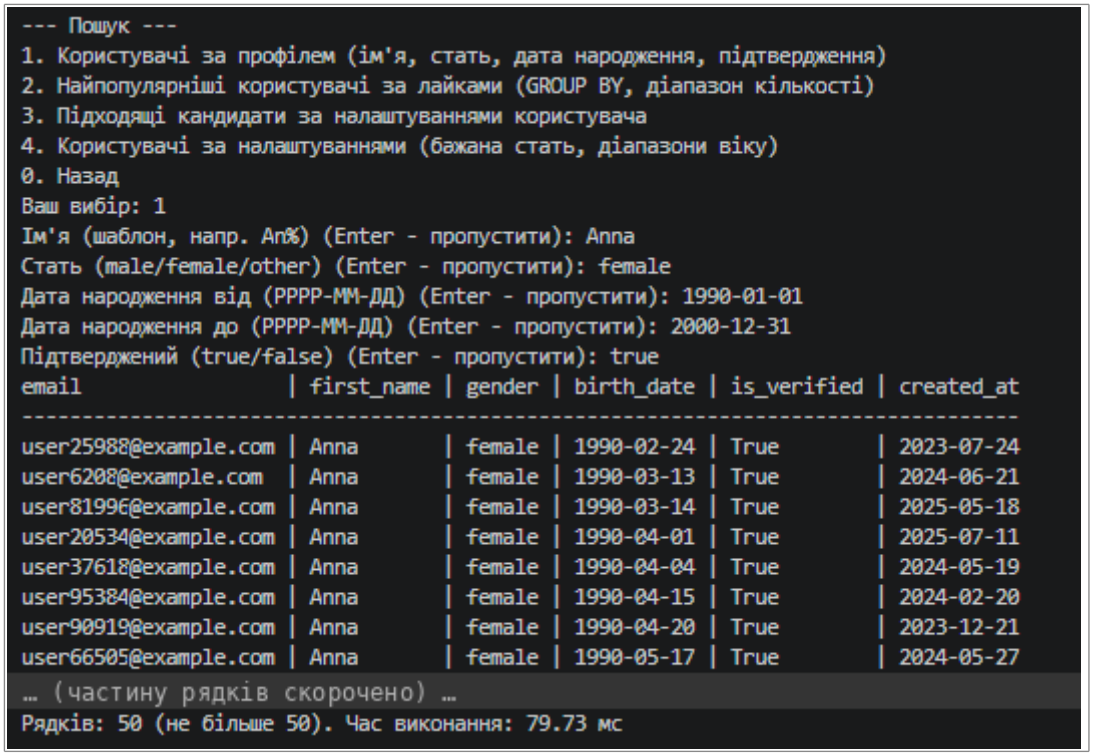
Запит 1. Користувачі за профілем

```
SELECT u.email, p.first_name, p.gender, p.birth_date, u.is_verified, u.created_at
FROM "User" u
JOIN "Profile" p ON p.user_id = u.user_id
WHERE (%(name)s::text IS NULL OR p.first_name ILIKE %(name)s)
  AND (%(gender)s::text IS NULL OR p.gender = %(gender)s)
  AND (%(born_from)s::date IS NULL OR p.birth_date >= %(born_from)s)
  AND (%(born_to)s::date IS NULL OR p.birth_date <= %(born_to)s)
  AND (%(verified)s::boolean IS NULL OR u.is_verified = %(verified)s)
ORDER BY p.birth_date, u.email
LIMIT %(lim)s
```

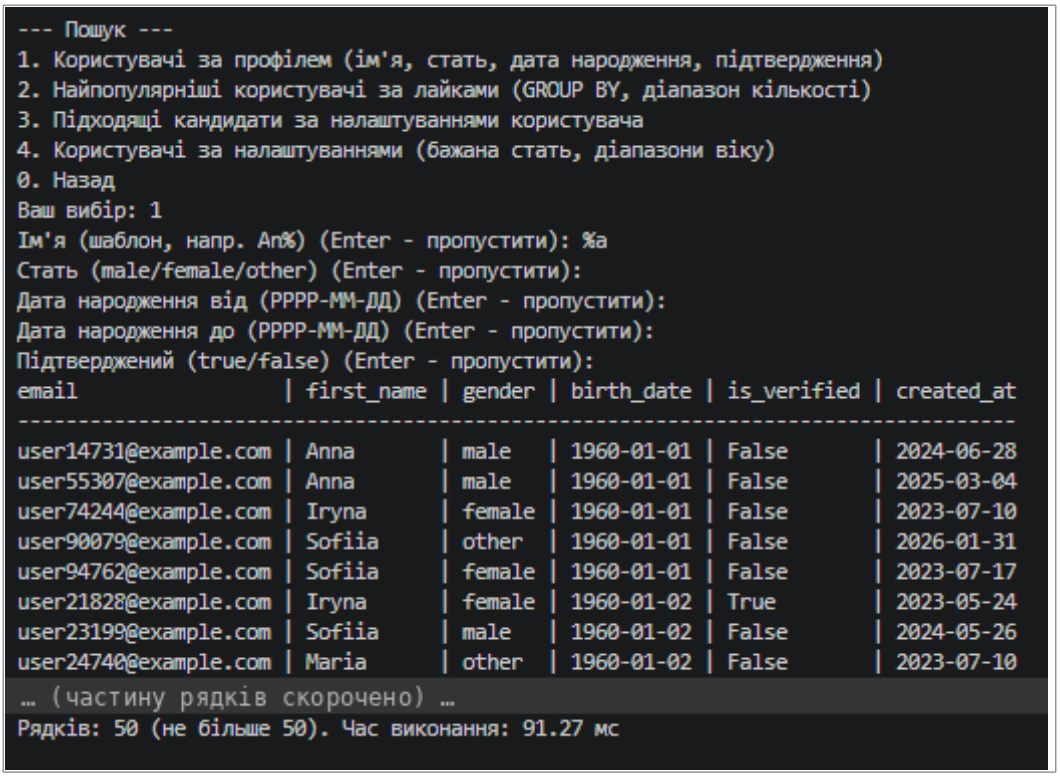
У прикладах довгі виводи скорочені: видно початок таблиці і останній рядок з кількістю рядків та часом.



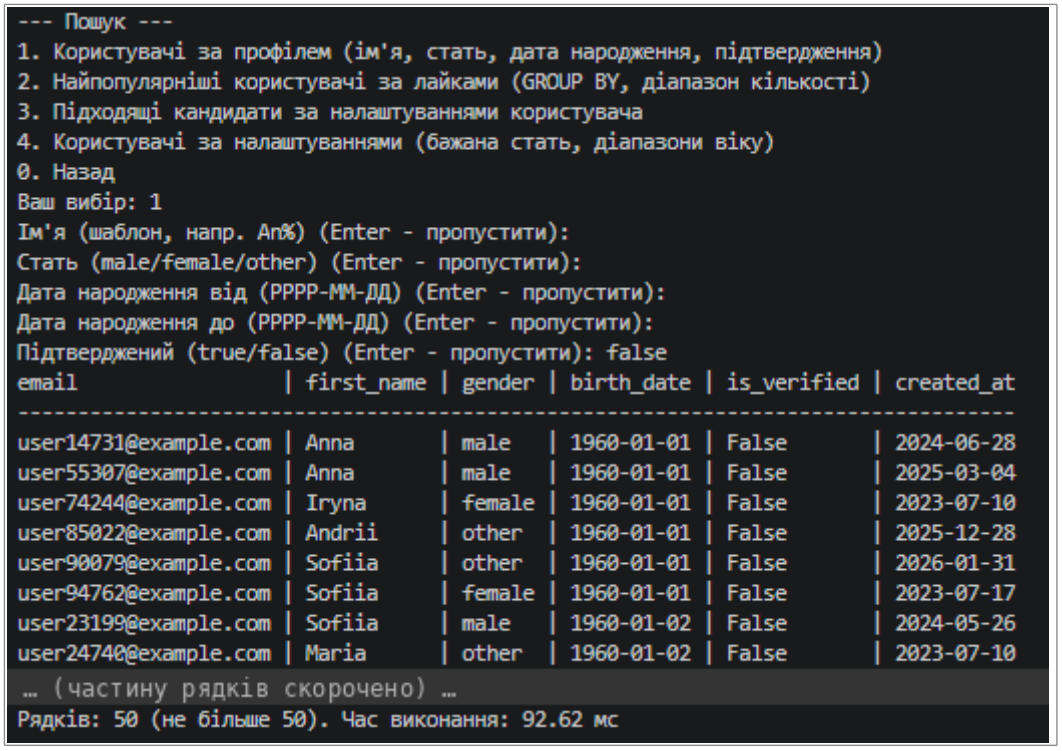
Запит 1: ім'я Anna, стать female, дата народження 1990-01-01 - 2000-12-31. 50 рядків, 84.20 мс



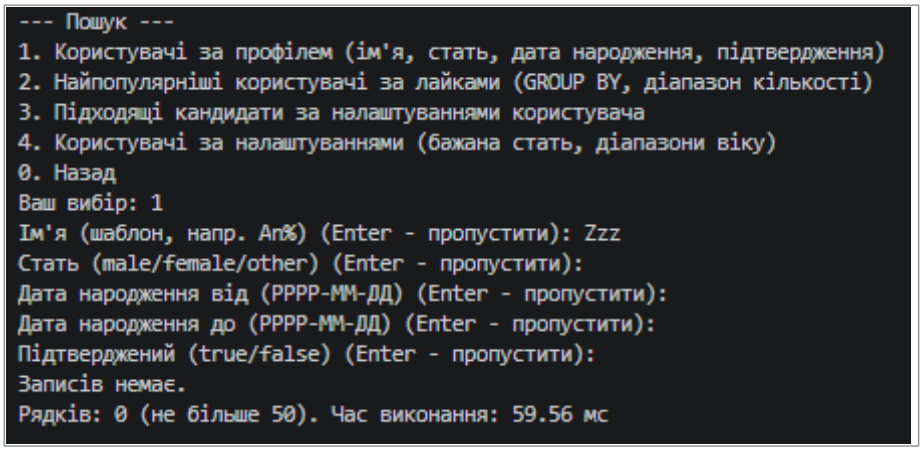
Запит 1 з умовою is_verified = true. 50 рядків, 79.73 мс



Запит 1: шаблон імені %a. 50 рядків, 91.27 мс



Запит 1: тільки is_verified = false. 50 рядків, 92.62 мс



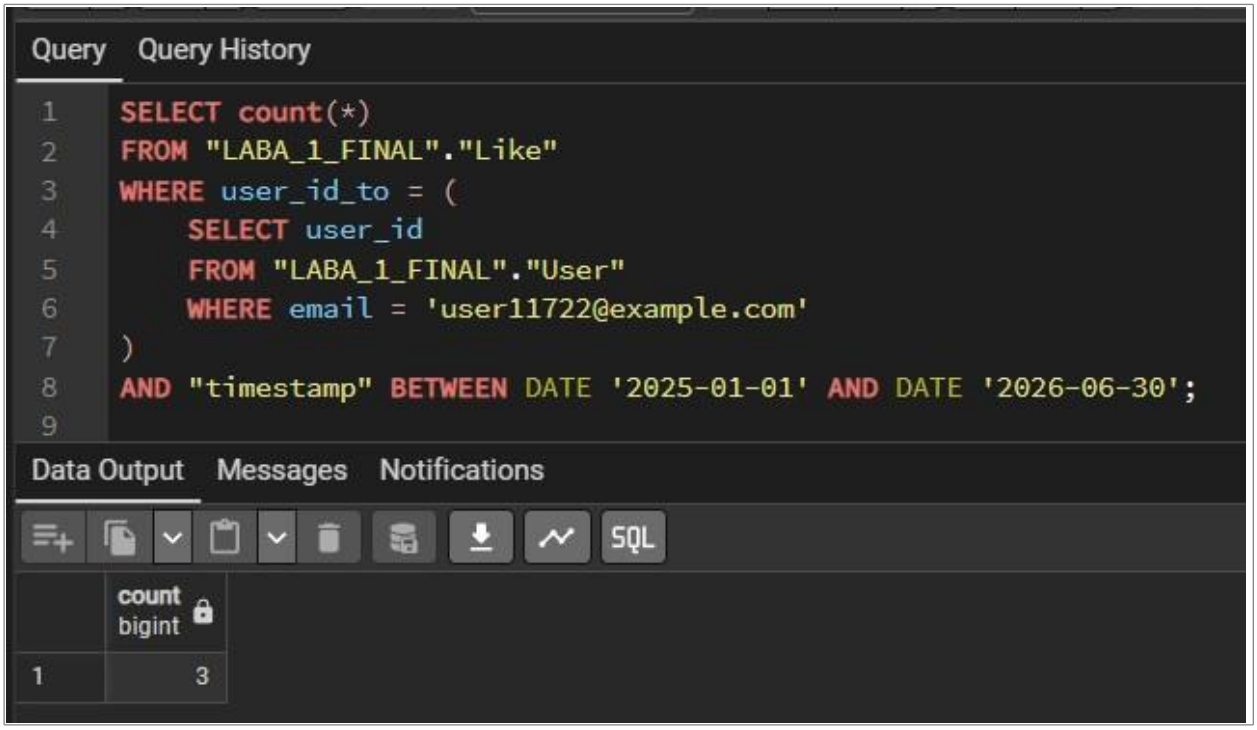
Запит 1: ім'я Zzz, нічого не знайдено. 0 рядків, 59.56 мс

Запит 2. Найпопулярніші користувачі (GROUP BY і HAVING)

```
SELECT p.first_name, u.email, COUNT(*) AS likes_received
FROM "Like" l
JOIN "User" u ON u.user_id = l.user_id_to
JOIN "Profile" p ON p.user_id = u.user_id
WHERE (%(d_from)s::date IS NULL OR l."timestamp" >= %(d_from)s)
  AND (%(d_to)s::date IS NULL OR l."timestamp" <= %(d_to)s)
  AND (%(name)s::text IS NULL OR p.first_name ILIKE %(name)s)
GROUP BY u.user_id, u.email, p.first_name
HAVING COUNT(*) >= %(min_likes)s
   AND (%(max_likes)s::int IS NULL OR COUNT(*) <= %(max_likes)s)
ORDER BY likes_received DESC, u.email
LIMIT %(lim)s
```

```
--- Пошук ---
1. Користувачі за профілем (ім'я, стать, дата народження, підтвердження)
2. Найпопулярніші користувачі за лайками (GROUP BY, діапазон кількості)
3. Підходящі кандидати за налаштуваннями користувача
4. Користувачі за налаштуваннями (бажана стать, діапазони віку)
0. Назад
Ваш вибір: 2
Лайки з дати (РРРР-ММ-ДД) (Enter - пропустити): 2025-01-01
Лайки до дати (РРРР-ММ-ДД) (Enter - пропустити): 2026-06-30
Ім'я (шаблон) (Enter - пропустити):
Мінімум лайків (Enter - пропустити): 2
Максимум лайків (Enter - пропустити): 3
first_name | email                | likes_received
-------------------------------------------------
Olena      | user10002@example.com | 3
Anna       | user10071@example.com | 3
Sofiia     | user101@example.com   | 3
Taras      | user10115@example.com | 3
Andrii     | user10170@example.com | 3
Andrii     | user10363@example.com | 3
Andrii     | user1038@example.com  | 3
… (частину рядків скорочено) …
Рядків: 50 (не більше 50). Час виконання: 187.13 мс
```

Запит 2: лайки з 2025-01-01 до 2026-06-30, кількість лайків від 2 до 3. 50 рядків, 187.13 мс



Перевірка: у таблиці Like для user11722@example.com за цей період рівно 3 лайки, як і в результаті пошуку

Запит 3. Підходящі кандидати

```
SELECT cand.email, p.first_name, p.gender,
       EXTRACT(YEAR FROM age(p.birth_date))::int AS age
FROM "User" me
JOIN "Preference" pr ON pr.user_id = me.user_id
CROSS JOIN "Profile" p
JOIN "User" cand ON cand.user_id = p.user_id
WHERE me.email = %(email)s
  AND p.user_id <> me.user_id
  AND (pr.target_gender IS NULL OR p.gender = pr.target_gender)
  AND (pr.min_age IS NULL OR EXTRACT(YEAR FROM age(p.birth_date)) >= pr.min_age)
  AND (pr.max_age IS NULL OR EXTRACT(YEAR FROM age(p.birth_date)) <= pr.max_age)
  AND NOT EXISTS (SELECT 1 FROM "Like" l
                  WHERE l.user_id_from = me.user_id AND l.user_id_to = p.user_id)
  AND (%(name)s::text IS NULL OR p.first_name ILIKE %(name)s)
ORDER BY age, cand.email
LIMIT %(lim)s
```

Query Query History

```
1  SELECT p.target_gender, p.min_age, p.max_age
2  FROM "LABA_1_FINAL"."Preference" p
3  JOIN "LABA_1_FINAL"."User" u ON u.user_id = p.user_id
4  WHERE u.email = 'user5@example.com';
5
```

Data Output Messages Notifications

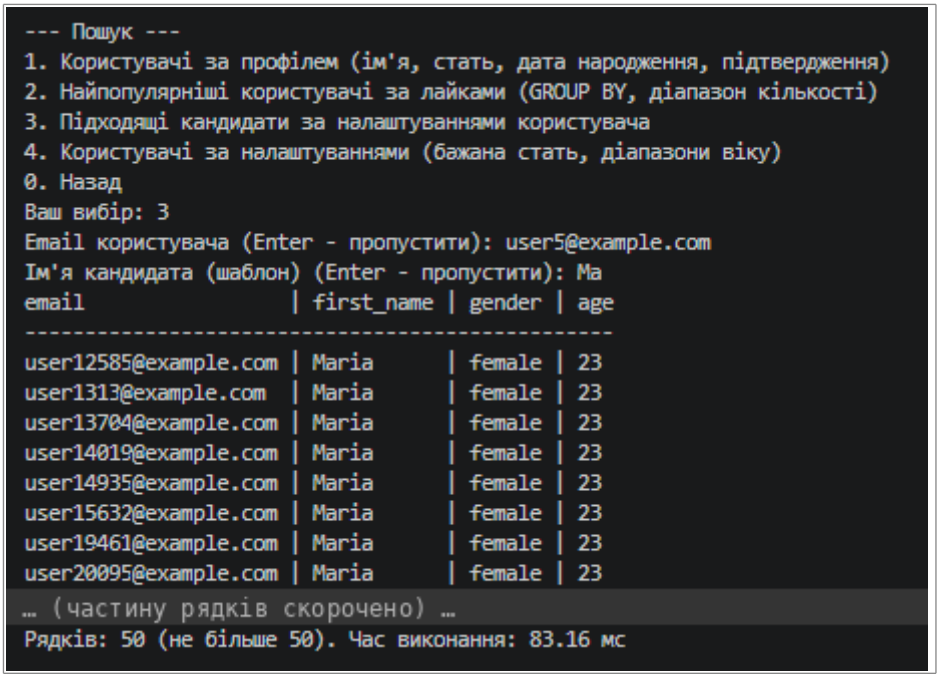
SQL

	target_gender character varying (10)	min_age integer	max_age integer
1	female	23	32

Налаштування user5@example.com у базі: шукає female, вік 23-32

```
--- Пошук ---
1. Користувачі за профілем (ім'я, стать, дата народження, підтвердження)
2. Найпопулярніші користувачі за лайками (GROUP BY, діапазон кількості)
3. Підходящі кандидати за налаштуваннями користувача
4. Користувачі за налаштуваннями (бажана стать, діапазони віку)
0. Назад
Ваш вибір: 3
Email користувача (Enter - пропустити): user5@example.com
Ім'я кандидата (шаблон) (Enter - пропустити):
email                | first_name | gender | age
-------------------------------------------------
user10011@example.com | Sofiia     | female | 23
user10072@example.com | Andrii     | female | 23
user10140@example.com | Iryna      | female | 23
user10181@example.com | Anna       | female | 23
user10331@example.com | Oleh       | female | 23
user10557@example.com | Anna       | female | 23
user10677@example.com | Iryna      | female | 23
user10929@example.com | Sofiia     | female | 23
 … (частину рядків скорочено) …
Рядків: 50 (не більше 50). Час виконання: 107.79 мс
```

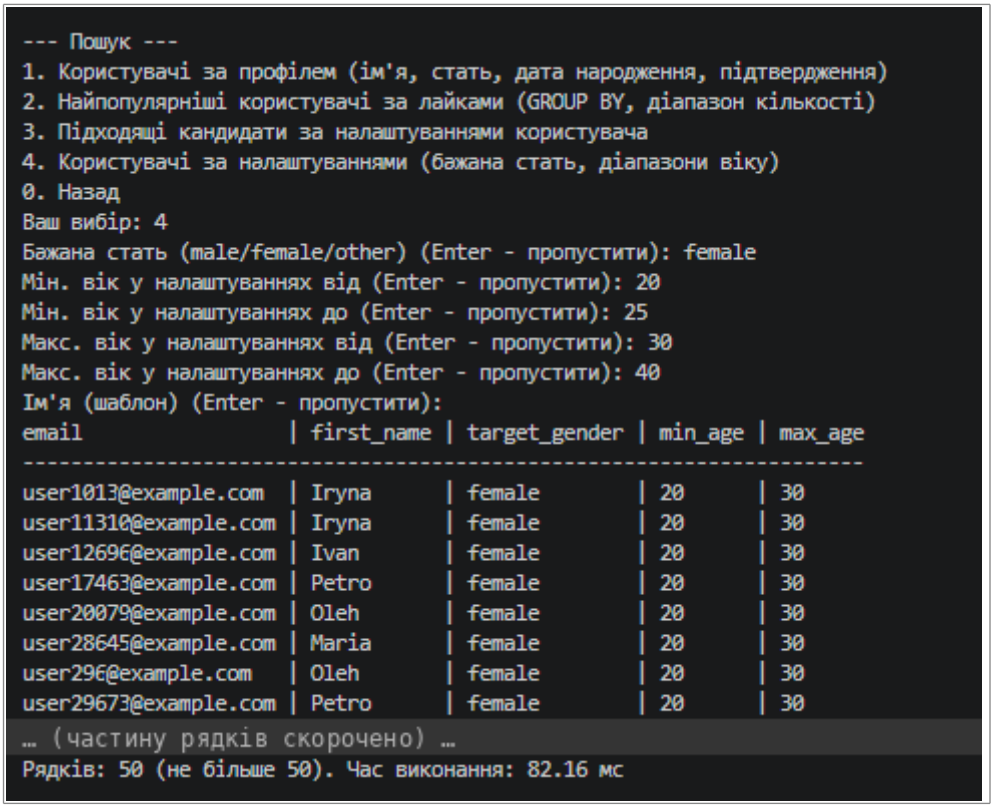
Запит 3 для user5@example.com. 50 рядків, 107.79 мс. Усі кандидати female, вік 23



Запит 3 з шаблоном імені Ma. Тільки Maria, 50 рядків, 83.16 мс

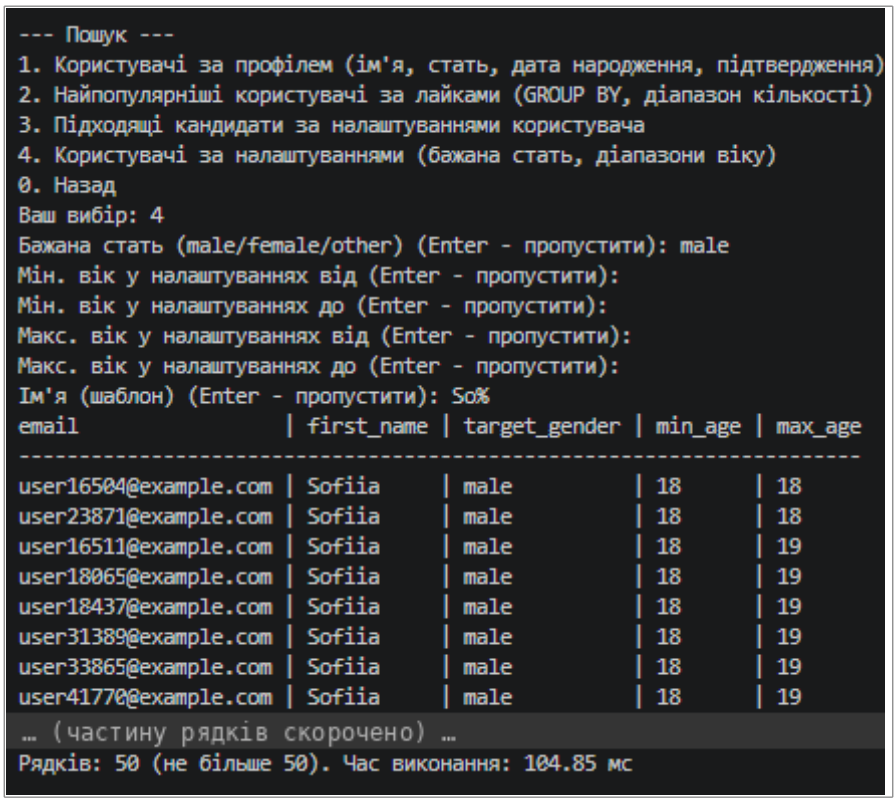
Запит 4. Користувачі за налаштуваннями

```
SELECT u.email, p.first_name, pr.target_gender, pr.min_age, pr.max_age
FROM "Preference" pr
JOIN "User" u ON u.user_id = pr.user_id
JOIN "Profile" p ON p.user_id = pr.user_id
WHERE (%(target)s::text IS NULL OR pr.target_gender = %(target)s)
  AND (%(min_from)s::int IS NULL OR pr.min_age >= %(min_from)s)
  AND (%(min_to)s::int IS NULL OR pr.min_age <= %(min_to)s)
  AND (%(max_from)s::int IS NULL OR pr.max_age >= %(max_from)s)
  AND (%(max_to)s::int IS NULL OR pr.max_age <= %(max_to)s)
  AND (%(name)s::text IS NULL OR p.first_name ILIKE %(name)s)
ORDER BY pr.min_age, pr.max_age, u.email
LIMIT %(lim)s
```



```
--- Пошук ---
1. Користувачі за профілем (ім'я, стать, дата народження, підтвердження)
2. Найпопулярніші користувачі за лайками (GROUP BY, діапазон кількості)
3. Підходящі кандидати за налаштуваннями користувача
4. Користувачі за налаштуваннями (бажана стать, діапазони віку)
0. Назад
Ваш вибір: 4
Бажана стать (male/female/other) (Enter - пропустити): female
Мін. вік у налаштуваннях від (Enter - пропустити): 20
Мін. вік у налаштуваннях до (Enter - пропустити): 25
Макс. вік у налаштуваннях від (Enter - пропустити): 30
Макс. вік у налаштуваннях до (Enter - пропустити): 40
Ім'я (шаблон) (Enter - пропустити):
email                | first_name | target_gender | min_age | max_age
-------------------------------------------------------------------
user1013@example.com  | Iryna      | female        | 20      | 30
user11310@example.com | Iryna      | female        | 20      | 30
user12696@example.com | Ivan       | female        | 20      | 30
user17463@example.com | Petro      | female        | 20      | 30
user20079@example.com | Oleh       | female        | 20      | 30
user28645@example.com | Maria      | female        | 20      | 30
user296@example.com   | Oleh       | female        | 20      | 30
user29673@example.com | Petro      | female        | 20      | 30
… (частину рядків скорочено) …
Рядків: 50 (не більше 50). Час виконання: 82.16 мс
```

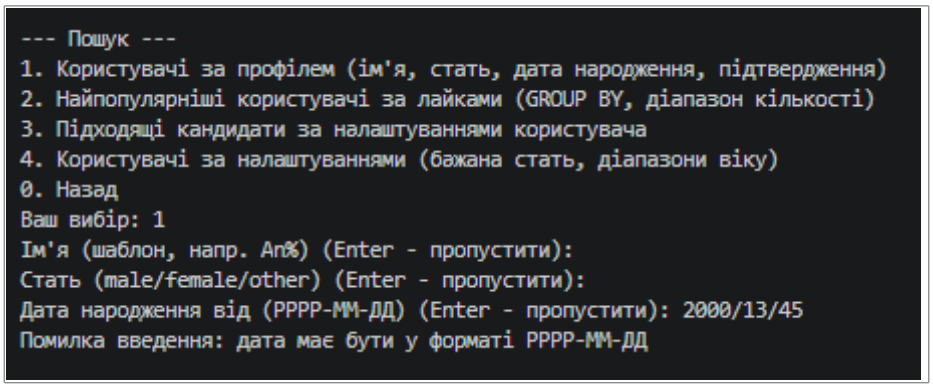
Запит 4: female, мінімальний вік 20-25, максимальний 30-40. 50 рядків, 82.16 мс



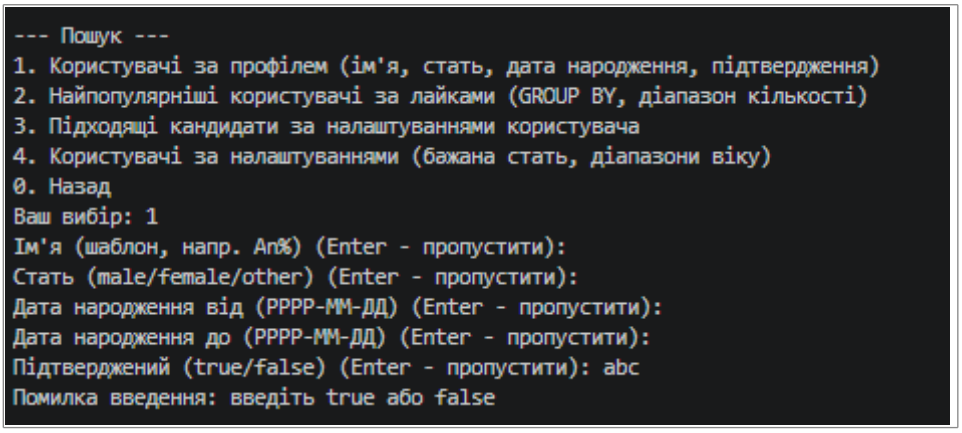
```
--- Пошук ---
1. Користувачі за профілем (ім'я, стать, дата народження, підтвердження)
2. Найпопулярніші користувачі за лайками (GROUP BY, діапазон кількості)
3. Підходящі кандидати за налаштуваннями користувача
4. Користувачі за налаштуваннями (бажана стать, діапазони віку)
0. Назад
Ваш вибір: 4
Бажана стать (male/female/other) (Enter - пропустити): male
Мін. вік у налаштуваннях від (Enter - пропустити):
Мін. вік у налаштуваннях до (Enter - пропустити):
Макс. вік у налаштуваннях від (Enter - пропустити):
Макс. вік у налаштуваннях до (Enter - пропустити):
Ім'я (шаблон) (Enter - пропустити): So%
email                | first_name | target_gender | min_age | max_age
-------------------------------------------------------------------
user16504@example.com | Sofiia     | male          | 18      | 18
user23871@example.com | Sofiia     | male          | 18      | 18
user16511@example.com | Sofiia     | male          | 18      | 19
user18065@example.com | Sofiia     | male          | 18      | 19
user18437@example.com | Sofiia     | male          | 18      | 19
user31389@example.com | Sofiia     | male          | 18      | 19
user33865@example.com | Sofiia     | male          | 18      | 19
user41770@example.com | Sofiia     | male          | 18      | 19
… (частину рядків скорочено) …
Рядків: 50 (не більше 50). Час виконання: 104.85 мс
```

Запит 4: male, ім'я So%. 50 рядків, 104.85 мс

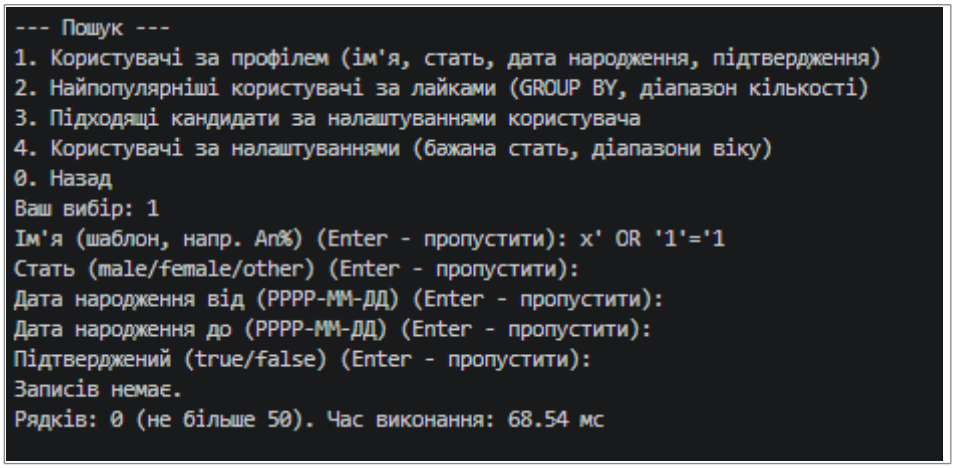
Помилки введення і спроба SQL-ін'єкції



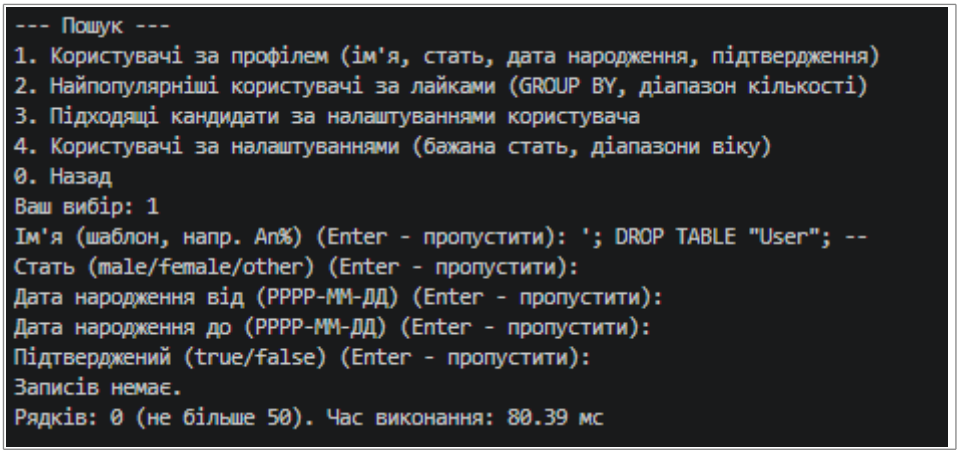
Неправильна дата у фільтрі



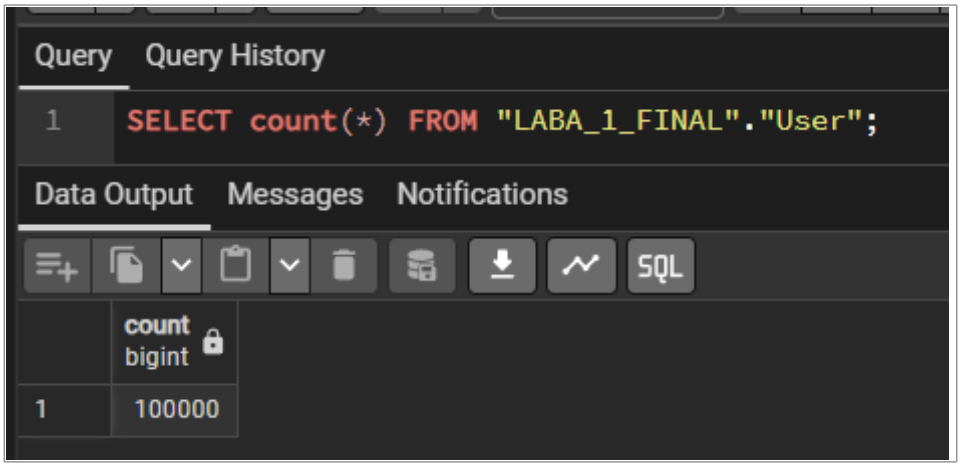
Неправильне значення true/false у фільтрі



В імені написав x' OR '1'='1. Програма шукає це як звичайний текст, знайдено 0 рядків, помилки немає



В імені написав '; DROP TABLE "User"; --. Теж 0 рядків і без помилки



Після цього таблиця User на місці, у ній 100 000 записів

Час виконання запитів

Запит	Параметри	Рядків	Час, мс
1	Anna, female, 1990-01-01 - 2000-12-31	50	84.20
1	те саме і is_verified = true	50	79.73
1	ім'я %a	50	91.27
1	is_verified = false	50	92.62
1	ім'я Zzz	0	59.56
1	дата народження 1999-01-01 - 1999-03-31 (без індексів)	50	86.91
2	лайки 2025-01-01 - 2026-06-30, кількість 2-3	50	187.13
3	user5@example.com	50	107.79
3	user5@example.com, ім'я Ma	50	83.16
4	female, мін. вік 20-25, макс. вік 30-40	50	82.16
4	male, ім'я So%	50	104.85

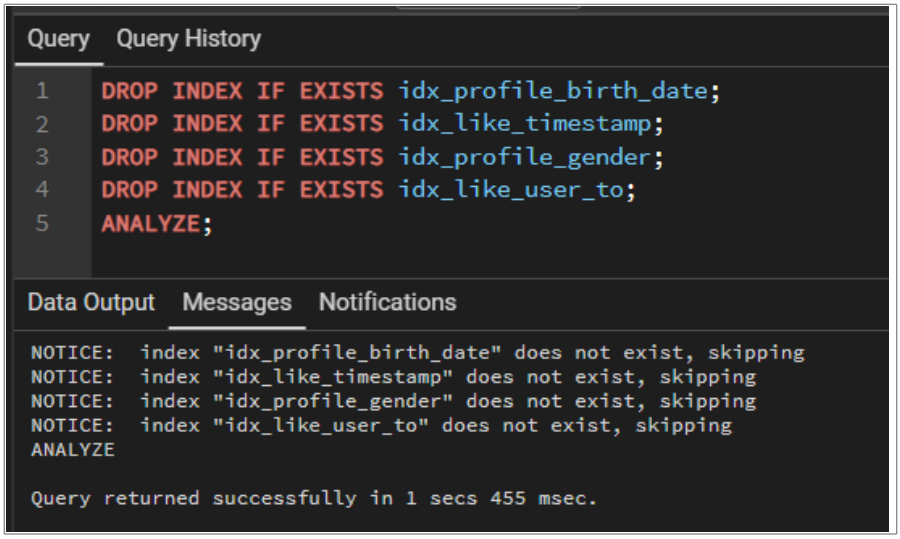
Виміри зроблені на 100 000 записів у User, Profile, Preference і 99 998 у Like. У час входить виконання запиту і отримання всіх рядків результату.

Швидкодія: без індексів і з індексами

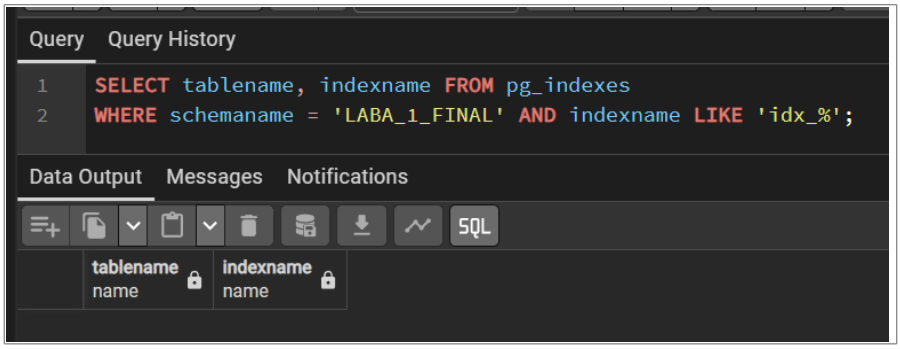
Швидкодію перевіряв для запитів 1, 2 і 3. Спочатку видалив індекси (sql/drop_indexes.sql) і виміряв час у програмі та через EXPLAIN ANALYZE в pgAdmin. Потім створив індекси (sql/indexes.sql) і виміряв ще раз з тими самими параметрами. Параметри: запит 1 - дата народження 1999-01-01 - 1999-03-31; запит 2 - лайки 2026-05-01 - 2026-06-30, мінімум 1; запит 3 - user5@example.com.

Час у програмі вимірюється навколо execute() і fetchall(), тому в нього входять планування запиту, виконання, передача рядків і обробка в Python. Через це його правильно порівнювати з сумою Planning Time і Execution Time з EXPLAIN, а не тільки з Execution Time.

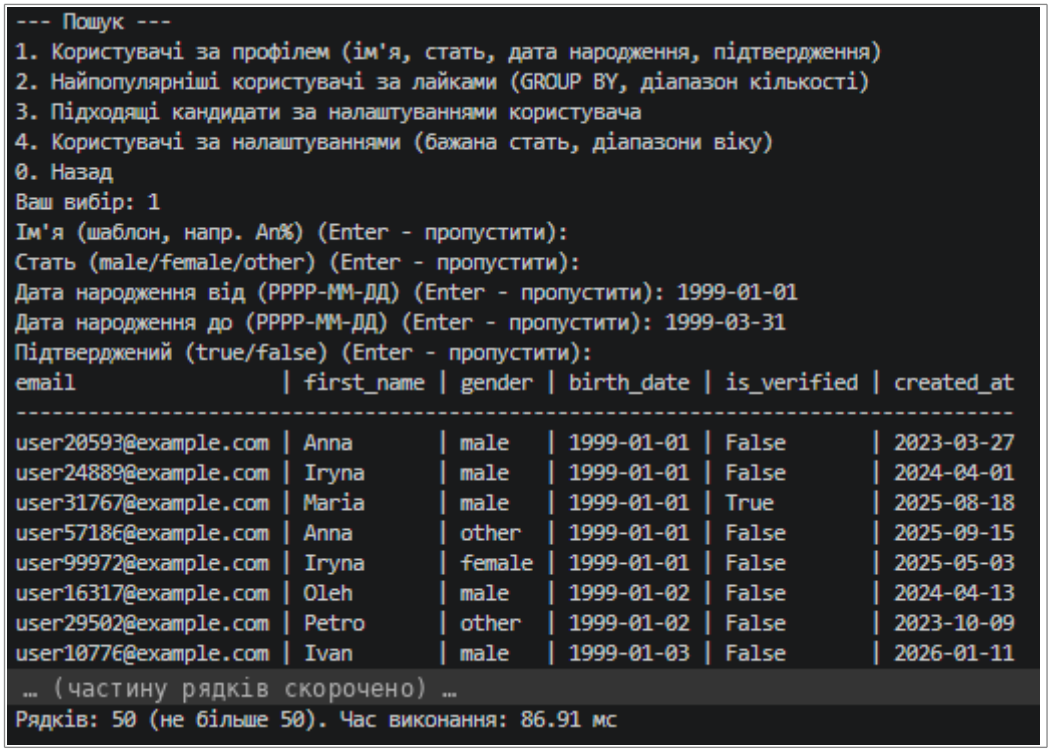
Без індексів



Видалення індексів (sql/drop_indexes.sql). Повідомлення «does not exist, skipping» означають, що індексів уже не було



Перевірка pg_indexes: індексів idx_% немає



Запит 1 у програмі без індексів: 86.91 мс

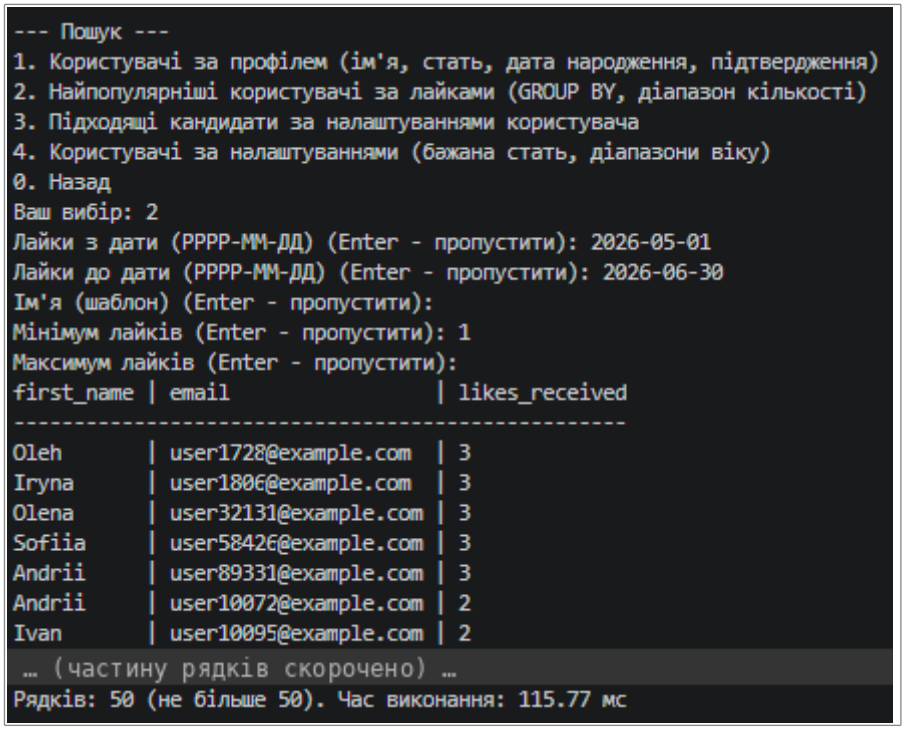
Query | Query History

```
1  EXPLAIN ANALYZE
2  SELECT u.email, p.first_name, p.gender, p.birth_date, u.is_verified, u.created_at
3  FROM "User" u
4  JOIN "Profile" p ON p.user_id = u.user_id
5  WHERE p.birth_date >= DATE '1999-01-01'
6    AND p.birth_date <= DATE '1999-03-31'
7  ORDER BY p.birth_date, u.email
8  LIMIT 50;
```

Data Output | Messages | Notifications

	QUERY PLAN text
4	Sort Key: p.birth_date, u.email
5	Presorted Key: p.birth_date
6	Full-sort Groups: 2 Sort Method: quicksort Average Memory: 27kB Peak Memory: 27kB
7	Buffers: shared hit=1273
8	-> Nested Loop (cost=2927.68..6625.43 rows=565 width=41) (actual time=79.505..85.812 rows=51.00 loops=1)
9	Buffers: shared hit=1273
10	-> Gather Merge (cost=2927.27..2991.66 rows=565 width=31) (actual time=79.460..85.365 rows=51.00 loops=1)
11	Workers Planned: 1
12	Workers Launched: 1
13	Buffers: shared hit=1069
14	-> Sort (cost=1927.26..1928.09 rows=332 width=31) (actual time=5.118..5.126 rows=25.50 loops=2)
15	Sort Key: p.birth_date
16	Sort Method: quicksort Memory: 53kB
17	Buffers: shared hit=1069
18	Worker 0: Sort Method: quicksort Memory: 25kB
19	-> Parallel Seq Scan on "Profile" p (cost=0.00..1913.35 rows=332 width=31) (actual time=0.033..4.915 rows=289.50 loo…
20	Filter: ((birth_date >= '1999-01-01'::date) AND (birth_date <= '1999-03-31'::date))
21	Rows Removed by Filter: 49710
22	Buffers: shared hit=1031
23	-> Index Scan using "User_pkey" on "User" u (cost=0.42..6.43 rows=1 width=42) (actual time=0.007..0.007 rows=1.00 loops=51)
24	Index Cond: (user_id = p.user_id)
25	Index Searches: 51
26	Buffers: shared hit=204
27	Planning:
28	Buffers: shared hit=16
29	Planning Time: 0.366 ms
30	Execution Time: 86.053 ms

EXPLAIN ANALYZE запиту 1 без індексів. Parallel Seq Scan по Profile, Planning Time 0.366 мс, Execution Time 86.053 мс



Запит 2 у програмі без індексів: 115.77 мс

Query | Query History

```
1  EXPLAIN ANALYZE
2  SELECT p.first_name, u.email, COUNT(*) AS likes_received
3  FROM "Like" l
4  JOIN "User" u ON u.user_id = l.user_id_to
5  JOIN "Profile" p ON p.user_id = u.user_id
6  WHERE l."timestamp" >= DATE '2026-05-01'
7    AND l."timestamp" <= DATE '2026-06-30'
8  GROUP BY u.user_id, u.email, p.first_name
9  HAVING COUNT(*) >= 1
10 ORDER BY likes_received DESC, u.email
11 LIMIT 50;
```

Data Output | Messages | Notifications

	QUERY PLAN text
15	Buffers: shared hit=23875
16	-> Nested Loop (cost=2354.95..6044.64 rows=3334 width=42) (actual time=26.081..52.343 rows=2726.50 loops=2)
17	Join Filter: (l.user_id_to = u.user_id)
18	Buffers: shared hit=23875
19	-> Parallel Hash Join (cost=2354.54..4276.62 rows=3334 width=37) (actual time=26.016..37.518 rows=2726.50 loops=2)
20	Hash Cond: (l.user_id_to = p.user_id)
21	Buffers: shared hit=2062
22	-> Parallel Seq Scan on "Like" l (cost=0.00..1913.34 rows=3334 width=16) (actual time=0.023..7.840 rows=2726.50 loops=2)
23	Filter: (("timestamp" >= '2026-05-01'::date) AND ("timestamp" <= '2026-06-30'::date))
24	Rows Removed by Filter: 47272
25	Buffers: shared hit=1031
26	-> Parallel Hash (cost=1619.24..1619.24 rows=58824 width=21) (actual time=25.472..25.473 rows=50000.00 loops=2)
27	Buckets: 131072 Batches: 1 Memory Usage: 6528kB
28	Buffers: shared hit=1031
29	-> Parallel Seq Scan on "Profile" p (cost=0.00..1619.24 rows=58824 width=21) (actual time=0.009..16.953 rows=100000.00 lo...
30	Buffers: shared hit=1031
31	-> Index Scan using "User_pkey" on "User" u (cost=0.42..0.52 rows=1 width=37) (actual time=0.005..0.005 rows=1.00 loops=5453)
32	Index Cond: (user_id = p.user_id)
33	Index Searches: 5453
34	Buffers: shared hit=21813
35	Planning:
36	Buffers: shared hit=32
37	Planning Time: 0.813 ms
38	Execution Time: 113.420 ms

EXPLAIN ANALYZE запиту 2 без індексів. Parallel Seq Scan по Like, Planning Time 0.813 мс, Execution Time 113.420 мс

```
--- Пошук ---
1. Користувачі за профілем (ім'я, стать, дата народження, підтвердження)
2. Найпопулярніші користувачі за лайками (GROUP BY, діапазон кількості)
3. Підходящі кандидати за налаштуваннями користувача
4. Користувачі за налаштуваннями (бажана стать, діапазони віку)
0. Назад
Ваш вибір: 3
Email користувача (Enter - пропустити): user5@example.com
Ім'я кандидата (шаблон) (Enter - пропустити):
email                | first_name | gender | age
-----------------------------------------------
user10011@example.com | Sofiia     | female | 23
user10072@example.com | Andrii     | female | 23
user10140@example.com | Iryna      | female | 23
user10181@example.com | Anna       | female | 23
user10331@example.com | Oleh       | female | 23
user10557@example.com | Anna       | female | 23
user10677@example.com | Iryna      | female | 23
user10929@example.com | Sofiia     | female | 23
… (частину рядків скорочено) …
Рядків: 50 (не більше 50). Час виконання: 114.73 мс
```

Запит 3 у програмі без індексів: 114.73 мс

Query | Query History

```sql
EXPLAIN ANALYZE
SELECT cand.email, p.first_name, p.gender,
       EXTRACT(YEAR FROM age(p.birth_date))::int AS age
FROM "User" me
JOIN "Preference" pr ON pr.user_id = me.user_id
CROSS JOIN "Profile" p
JOIN "User" cand ON cand.user_id = p.user_id
WHERE me.email = 'user5@example.com'
  AND p.user_id <> me.user_id
  AND (pr.target_gender IS NULL OR p.gender = pr.target_gender)
  AND (pr.min_age IS NULL OR EXTRACT(YEAR FROM age(p.birth_date)) >= pr.min_age)
  AND (pr.max_age IS NULL OR EXTRACT(YEAR FROM age(p.birth_date)) <= pr.max_age)
  AND NOT EXISTS (SELECT 1 FROM "Like" l
                  WHERE l.user_id_from = me.user_id AND l.user_id_to = p.user_id)
ORDER BY age, cand.email
LIMIT 50;
```

Data Output | Messages | Notifications

SQL | Show

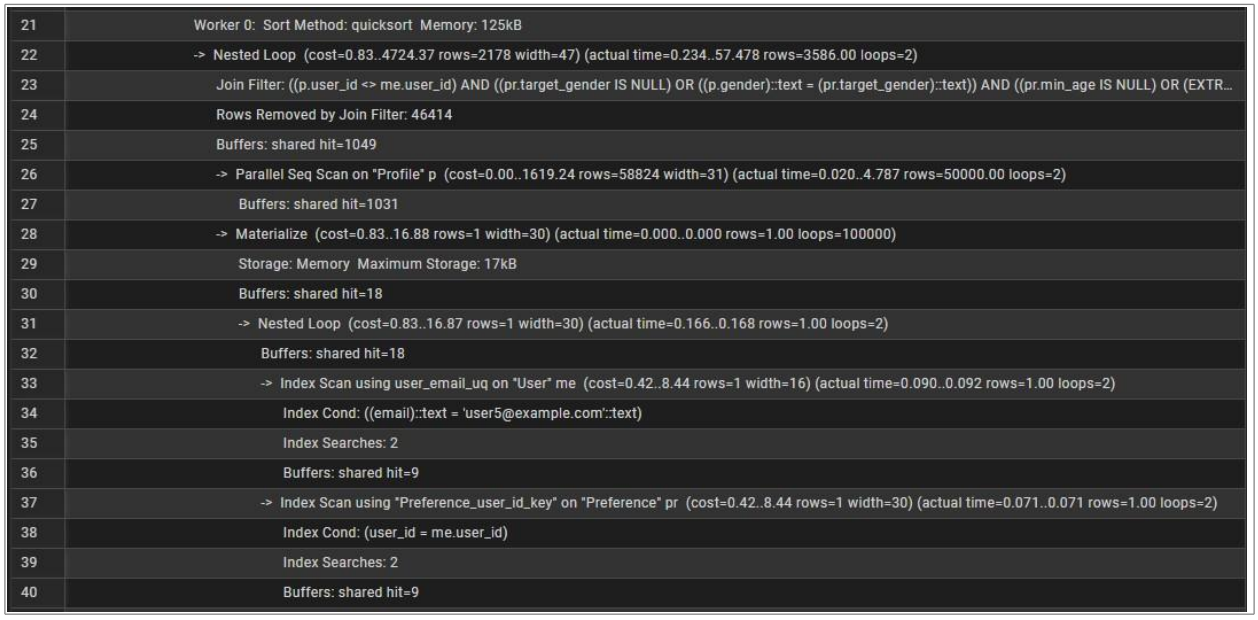
QUERY PLAN
text

Запит 3: текст EXPLAIN ANALYZE



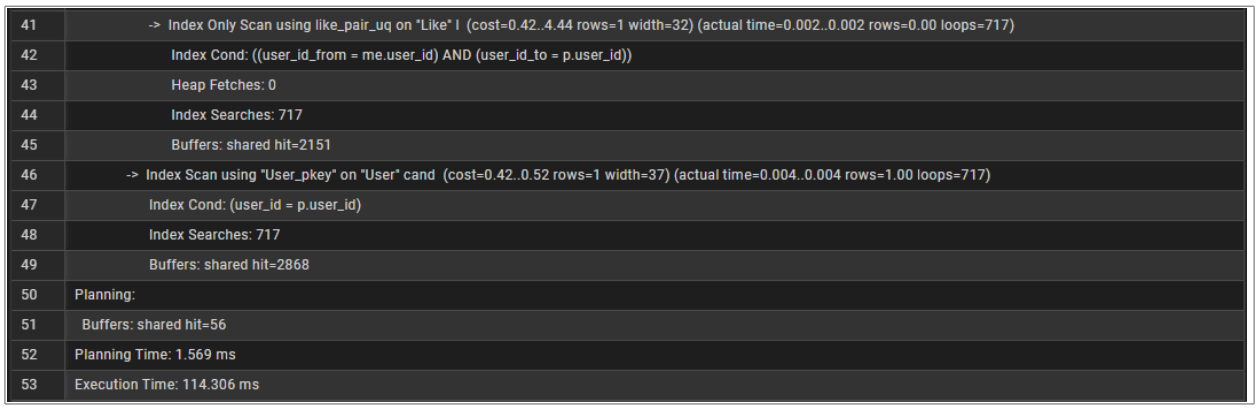
```
1  Limit  (cost=5852.91..6201.30 rows=50 width=36) (actual time=106.980..113.986 rows=50.00 loops=1)
2    Buffers: shared hit=6075
3    ->  Incremental Sort  (cost=5852.91..24234.12 rows=2638 width=36) (actual time=106.979..113.979 rows=50.00 loops=1)
4          Sort Key: ((EXTRACT(year FROM age((CURRENT_DATE)::timestamp with time zone, (p.birth_date)::timestamp with time zone)))::integer), cand.email
5          Presorted Key: ((EXTRACT(year FROM age((CURRENT_DATE)::timestamp with time zone, (p.birth_date)::timestamp with time zone)))::integer)
6          Full-sort Groups: 1  Sort Method: quicksort  Average Memory: 29kB  Peak Memory: 29kB
7          Pre-sorted Groups: 1  Sort Method: top-N heapsort  Average Memory: 30kB  Peak Memory: 30kB
8          Buffers: shared hit=6075
9          ->  Nested Loop  (cost=5845.97..24115.41 rows=2638 width=36) (actual time=99.666..112.465 rows=717.00 loops=1)
10               Buffers: shared hit=6075
11               ->  Nested Loop Anti Join  (cost=5845.55..22709.88 rows=2638 width=31) (actual time=99.642..108.747 rows=717.00 loops=1)
12                     Buffers: shared hit=3207
13                     ->  Gather Merge  (cost=5845.13..6267.17 rows=3703 width=47) (actual time=99.611..107.002 rows=717.00 loops=1)
14                           Workers Planned: 1
15                           Workers Launched: 1
16                           Buffers: shared hit=1056
17                           ->  Sort  (cost=4845.12..4850.57 rows=2178 width=47) (actual time=59.372..59.493 rows=717.00 loops=2)
18                                 Sort Key: ((EXTRACT(year FROM age((CURRENT_DATE)::timestamp with time zone, (p.birth_date)::timestamp with time zone)))::integer)
19                                 Sort Method: quicksort  Memory: 621kB
20                                 Buffers: shared hit=1056
```

План запиту 3 без індексів, початок



```
21                           Worker 0:  Sort Method: quicksort  Memory: 125kB
22                           ->  Nested Loop  (cost=0.83..4724.37 rows=2178 width=47) (actual time=0.234..57.478 rows=3586.00 loops=2)
23                                 Join Filter: ((p.user_id <> me.user_id) AND ((pr.target_gender IS NULL) OR ((p.gender)::text = (pr.target_gender)::text)) AND ((pr.min_age IS NULL) OR (EXTR...
24                                 Rows Removed by Join Filter: 46414
25                                 Buffers: shared hit=1049
26                                 ->  Parallel Seq Scan on "Profile" p  (cost=0.00..1619.24 rows=58824 width=31) (actual time=0.020..4.787 rows=50000.00 loops=2)
27                                       Buffers: shared hit=1031
28                                 ->  Materialize  (cost=0.83..16.88 rows=1 width=30) (actual time=0.000..0.000 rows=1.00 loops=100000)
29                                       Storage: Memory  Maximum Storage: 17kB
30                                       Buffers: shared hit=18
31                                       ->  Nested Loop  (cost=0.83..16.87 rows=1 width=30) (actual time=0.166..0.168 rows=1.00 loops=2)
32                                             Buffers: shared hit=18
33                                             ->  Index Scan using user_email_uq on "User" me  (cost=0.42..8.44 rows=1 width=16) (actual time=0.090..0.092 rows=1.00 loops=2)
34                                                   Index Cond: ((email)::text = 'user5@example.com'::text)
35                                                   Index Searches: 2
36                                                   Buffers: shared hit=9
37                                             ->  Index Scan using "Preference_user_id_key" on "Preference" pr  (cost=0.42..8.44 rows=1 width=30) (actual time=0.071..0.071 rows=1.00 loops=2)
38                                                   Index Cond: (user_id = me.user_id)
39                                                   Index Searches: 2
40                                                   Buffers: shared hit=9
```

План запиту 3 без індексів, середина. Parallel Seq Scan по Profile, user_email_uq і Preference_user_id_key



```
41                                 ->  Index Only Scan using like_pair_uq on "Like" l  (cost=0.42..4.44 rows=1 width=32) (actual time=0.002..0.002 rows=0.00 loops=717)
42                                       Index Cond: ((user_id_from = me.user_id) AND (user_id_to = p.user_id))
43                                       Heap Fetches: 0
44                                       Index Searches: 717
45                                       Buffers: shared hit=2151
46                           ->  Index Scan using "User_pkey" on "User" cand  (cost=0.42..0.52 rows=1 width=37) (actual time=0.004..0.004 rows=1.00 loops=717)
47                                 Index Cond: (user_id = p.user_id)
48                                 Index Searches: 717
49                                 Buffers: shared hit=2868
50 Planning:
51   Buffers: shared hit=56
52 Planning Time: 1.569 ms
53 Execution Time: 114.306 ms
```

План запиту 3 без індексів, кінець. Planning Time 1.569 мс, Execution Time 114.306 мс

Створення індексів

Для запитів 1 і 2 створив індекси по стовпцях, які є в умовах з діапазоном. Для запиту 3 створив індекс по статі, щоб показати, що не кожен індекс прискорює запит: стать має лише три значення, кожне охоплює близько третини таблиці. Умови за віком рахуються через age(p.birth_date), а ця функція залежить від поточної дати, тому індекс по такому виразу створити не можна. Інші умови запиту 3 (пошук за email, з'єднання з Preference, перевірка лайків) уже використовують унікальні індекси, які є в схемі, це видно в плані.

```
-- запит 1
CREATE INDEX IF NOT EXISTS idx_profile_birth_date ON "Profile"(birth_date);
-- запит 2
CREATE INDEX IF NOT EXISTS idx_like_timestamp ON "Like"("timestamp");
-- запит 3
CREATE INDEX IF NOT EXISTS idx_profile_gender ON "Profile"(gender);
ANALYZE;
```



Створення індексів (sql/indexes.sql)

З індексами

```
--- Пошук ---
1. Користувачі за профілем (ім'я, стать, дата народження, підтвердження)
2. Найпопулярніші користувачі за лайками (GROUP BY, діапазон кількості)
3. Підходящі кандидати за налаштуваннями користувача
4. Користувачі за налаштуваннями (бажана стать, діапазони віку)
0. Назад
Ваш вибір: 1
Ім'я (шаблон, напр. An%) (Enter - пропустити):
Стать (male/female/other) (Enter - пропустити):
Дата народження від (РРРР-ММ-ДД) (Enter - пропустити): 1999-01-01
Дата народження до (РРРР-ММ-ДД) (Enter - пропустити): 1999-03-31
Підтверджений (true/false) (Enter - пропустити):
email                | first_name | gender | birth_date | is_verified | created_at
-------------------------------------------------------------------------------
user20593@example.com | Anna       | male   | 1999-01-01 | False       | 2023-03-27
user24889@example.com | Iryna      | male   | 1999-01-01 | False       | 2024-04-01
user31767@example.com | Maria      | male   | 1999-01-01 | True        | 2025-08-18
user57186@example.com | Anna       | other  | 1999-01-01 | False       | 2025-09-15
user99972@example.com | Iryna      | female | 1999-01-01 | False       | 2025-05-03
user16317@example.com | Oleh       | male   | 1999-01-02 | False       | 2024-04-13
user29502@example.com | Petro      | other  | 1999-01-02 | False       | 2023-10-09
user10776@example.com | Ivan       | male   | 1999-01-03 | False       | 2026-01-11
… (частину рядків скорочено) …
Рядків: 50 (не більше 50). Час виконання: 2.21 мс
```

Запит 1 у програмі з індексами: 2.21 мс

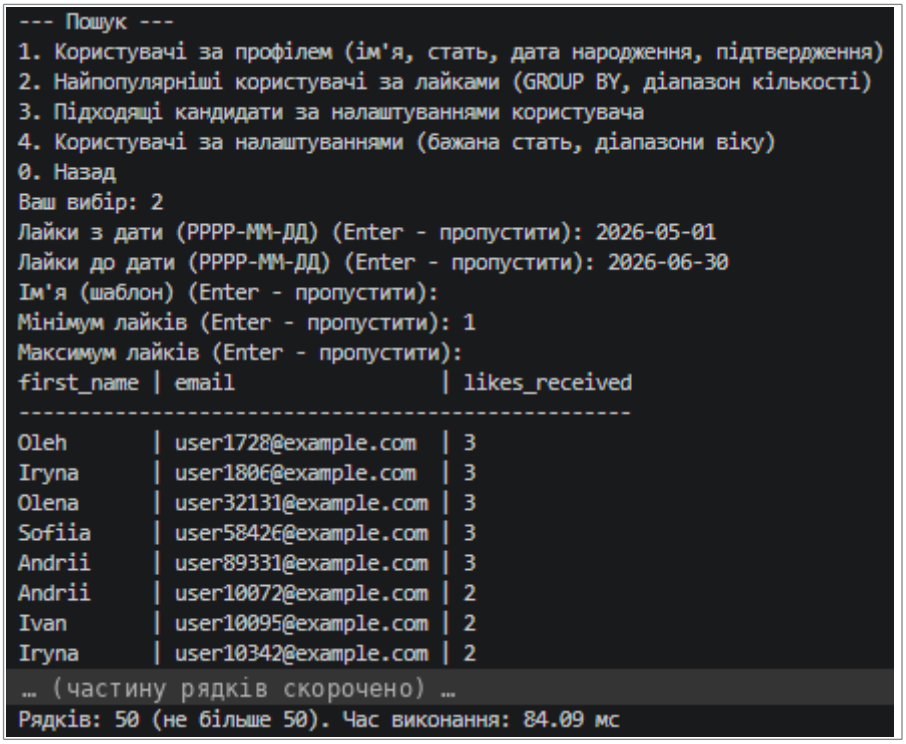
Query | Query History

```
1  EXPLAIN ANALYZE
2  SELECT u.email, p.first_name, p.gender, p.birth_date, u.is_verified, u.created_at
3  FROM "User" u
4  JOIN "Profile" p ON p.user_id = u.user_id
5  WHERE p.birth_date >= DATE '1999-01-01'
6    AND p.birth_date <= DATE '1999-03-31'
7  ORDER BY p.birth_date, u.email
8  LIMIT 50;
```

Data Output | Messages | Notifications

	QUERY PLAN text
1	Limit (cost=10.42..489.43 rows=50 width=41) (actual time=0.400..0.513 rows=50.00 loops=1)
2	Buffers: shared hit=255 read=2
3	-> Incremental Sort (cost=10.42..5557.38 rows=579 width=41) (actual time=0.399..0.506 rows=50.00 loops=1)
4	Sort Key: p.birth_date, u.email
5	Presorted Key: p.birth_date
6	Full-sort Groups: 2 Sort Method: quicksort Average Memory: 27kB Peak Memory: 27kB
7	Buffers: shared hit=255 read=2
8	-> Nested Loop (cost=0.71..5531.64 rows=579 width=41) (actual time=0.100..0.407 rows=51.00 loops=1)
9	Buffers: shared hit=255 read=2
10	-> Index Scan using idx_profile_birth_date on "Profile" p (cost=0.29..1827.77 rows=579 width=31) (actual time=0.083..0.140 rows=51.00 lo...
11	Index Cond: ((birth_date >= '1999-01-01'::date) AND (birth_date <= '1999-03-31'::date))
12	Index Searches: 1
13	Buffers: shared hit=51 read=2
14	-> Index Scan using "User_pkey" on "User" u (cost=0.42..6.40 rows=1 width=42) (actual time=0.005..0.005 rows=1.00 loops=51)
15	Index Cond: (user_id = p.user_id)
16	Index Searches: 51
17	Buffers: shared hit=204
18	Planning:
19	Buffers: shared hit=97 read=2
20	Planning Time: 1.997 ms
21	Execution Time: 0.553 ms

EXPLAIN ANALYZE запиту 1 з індексами. Index Scan по idx_profile_birth_date, Execution Time 0.553 мс (у плані є read=2, це перший запит після створення індексу)



Запит 2 у програмі з індексами: 84.09 мс

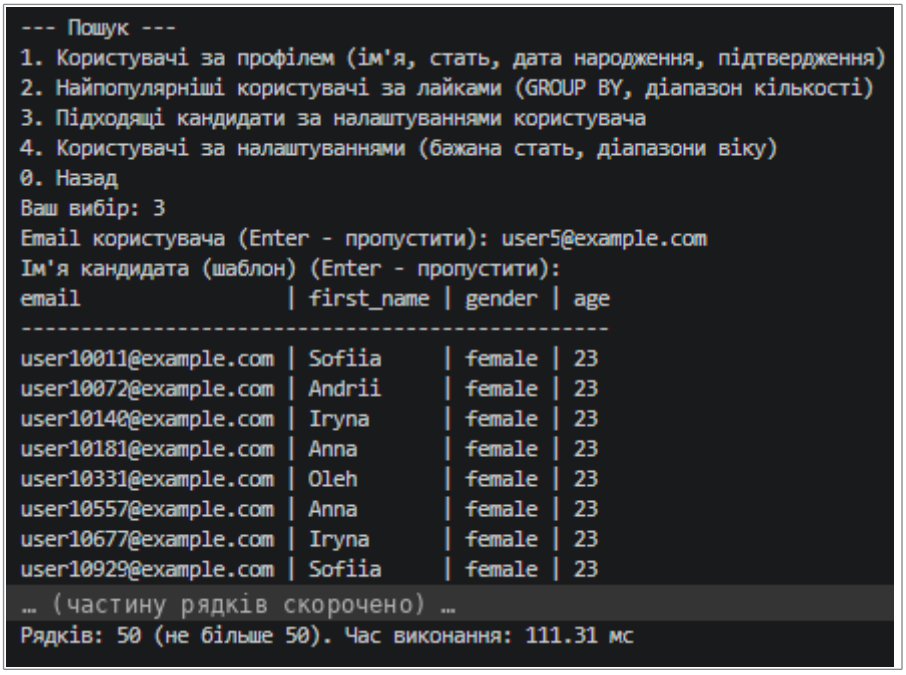
Query | Query History

```sql
EXPLAIN ANALYZE
SELECT p.first_name, u.email, COUNT(*) AS likes_received
FROM "Like" l
JOIN "User" u ON u.user_id = l.user_id_to
JOIN "Profile" p ON p.user_id = u.user_id
WHERE l."timestamp" >= DATE '2026-05-01'
  AND l."timestamp" <= DATE '2026-06-30'
GROUP BY u.user_id, u.email, p.first_name
HAVING COUNT(*) >= 1
ORDER BY likes_received DESC, u.email
LIMIT 50;
```

Data Output | Messages | Notifications

	QUERY PLAN text
19	-> Parallel Hash Join (cost=1192.49..3050.81 rows=3143 width=37) (actual time=2.348..15.506 rows=2726.50 loops=2)
20	Hash Cond: (p.user_id = l.user_id_to)
21	Buffers: shared hit=2066
22	-> Parallel Seq Scan on "Profile" p (cost=0.00..1619.24 rows=58824 width=21) (actual time=0.005..7.553 rows=100000.00 loops=1)
23	Buffers: shared hit=1031
24	-> Parallel Hash (cost=1153.20..1153.20 rows=3143 width=16) (actual time=2.267..2.269 rows=2726.50 loops=2)
25	Buckets: 8192 Batches: 1 Memory Usage: 320kB
26	Buffers: shared hit=1035
27	-> Parallel Bitmap Heap Scan on "Like" l (cost=75.06..1153.20 rows=3143 width=16) (actual time=0.678..3.285 rows=5453.00 loops...
28	Recheck Cond: (("timestamp" >= '2026-05-01'::date) AND ("timestamp" <= '2026-06-30'::date))
29	Heap Blocks: exact=1028
30	Buffers: shared hit=1035
31	-> Bitmap Index Scan on idx_like_timestamp (cost=0.00..73.72 rows=5343 width=0) (actual time=0.477..0.478 rows=5453.00 lo...
32	Index Cond: (("timestamp" >= '2026-05-01'::date) AND ("timestamp" <= '2026-06-30'::date))
33	Index Searches: 1
34	Buffers: shared hit=7
35	-> Index Scan using "User_pkey" on "User" u (cost=0.42..0.52 rows=1 width=37) (actual time=0.003..0.003 rows=1.00 loops=5453)
36	Index Cond: (user_id = p.user_id)
37	Index Searches: 5453
38	Buffers: shared hit=21812
39	Planning:
40	Buffers: shared hit=35
41	Planning Time: 1.399 ms
42	Execution Time: 81.826 ms

EXPLAIN ANALYZE запиту 2 з індексами. Bitmap Index Scan по idx_like_timestamp, Execution Time 81.826 мс



Запит 3 у програмі з індексами: 111.31 мс

Query Query History

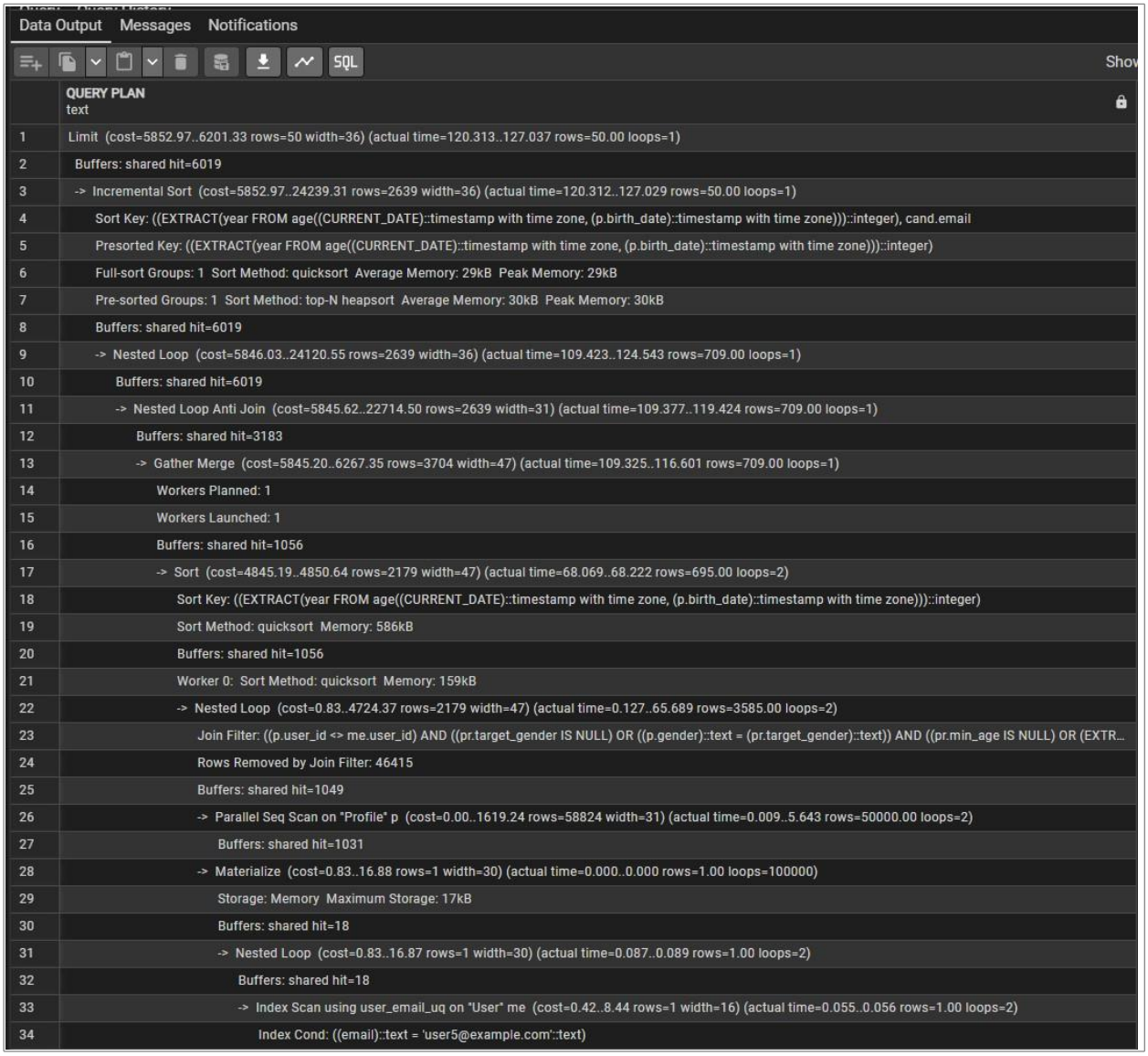
```
1   EXPLAIN ANALYZE
2   SELECT cand.email, p.first_name, p.gender,
3          EXTRACT(YEAR FROM age(p.birth_date))::int AS age
4   FROM "User" me
5   JOIN "Preference" pr ON pr.user_id = me.user_id
6   CROSS JOIN "Profile" p
7   JOIN "User" cand ON cand.user_id = p.user_id
8   WHERE me.email = 'user5@example.com'
9     AND p.user_id <> me.user_id
10    AND (pr.target_gender IS NULL OR p.gender = pr.target_gender)
11    AND (pr.min_age IS NULL OR EXTRACT(YEAR FROM age(p.birth_date)) >= pr.min_age)
12    AND (pr.max_age IS NULL OR EXTRACT(YEAR FROM age(p.birth_date)) <= pr.max_age)
13    AND NOT EXISTS (SELECT 1 FROM "Like" l
14                    WHERE l.user_id_from = me.user_id AND l.user_id_to = p.user_id)
15  ORDER BY age, cand.email
16  LIMIT 50;
```

Data Output Messages Notifications

SQL Show

	QUERY PLAN text
34	Index Cond: ((email)::text = 'user5@example.com'::text)
35	Index Searches: 2
36	Buffers: shared hit=9
37	-> Index Scan using "Preference_user_id_key" on "Preference" pr (cost=0.42..8.44 rows=1 width=30) (actual time=0.041..0.041 rows=1.00 loops=2)
38	Index Cond: (user_id = me.user_id)
39	Index Searches: 2
40	Buffers: shared hit=9
41	-> Index Only Scan using like_pair_uq on "Like" l (cost=0.42..4.44 rows=1 width=32) (actual time=0.002..0.002 rows=0.00 loops=717)
42	Index Cond: ((user_id_from = me.user_id) AND (user_id_to = p.user_id))
43	Heap Fetches: 0
44	Index Searches: 717
45	Buffers: shared hit=2151
46	-> Index Scan using "User_pkey" on "User" cand (cost=0.42..0.52 rows=1 width=37) (actual time=0.004..0.004 rows=1.00 loops=717)
47	Index Cond: (user_id = p.user_id)
48	Index Searches: 717
49	Buffers: shared hit=2868
50	Planning:
51	Buffers: shared hit=56
52	Planning Time: 1.522 ms
53	Execution Time: 107.783 ms

EXPLAIN ANALYZE запиту 3 з індексами (нижня частина плану). Planning Time 1.522 мс, Execution Time 107.783 мс



```
Data Output  Messages  Notifications
QUERY PLAN
text
Limit  (cost=5852.97..6201.33 rows=50 width=36) (actual time=120.313..127.037 rows=50.00 loops=1)
 Buffers: shared hit=6019
 -> Incremental Sort  (cost=5852.97..24239.31 rows=2639 width=36) (actual time=120.312..127.029 rows=50.00 loops=1)
    Sort Key: ((EXTRACT(year FROM age((CURRENT_DATE)::timestamp with time zone, (p.birth_date)::timestamp with time zone)))::integer), cand.email
    Presorted Key: ((EXTRACT(year FROM age((CURRENT_DATE)::timestamp with time zone, (p.birth_date)::timestamp with time zone)))::integer)
    Full-sort Groups: 1  Sort Method: quicksort  Average Memory: 29kB  Peak Memory: 29kB
    Pre-sorted Groups: 1  Sort Method: top-N heapsort  Average Memory: 30kB  Peak Memory: 30kB
    Buffers: shared hit=6019
    -> Nested Loop  (cost=5846.03..24120.55 rows=2639 width=36) (actual time=109.423..124.543 rows=709.00 loops=1)
       Buffers: shared hit=6019
       -> Nested Loop Anti Join  (cost=5845.62..22714.50 rows=2639 width=31) (actual time=109.377..119.424 rows=709.00 loops=1)
          Buffers: shared hit=3183
          -> Gather Merge  (cost=5845.20..6267.35 rows=3704 width=47) (actual time=109.325..116.601 rows=709.00 loops=1)
             Workers Planned: 1
             Workers Launched: 1
             Buffers: shared hit=1056
             -> Sort  (cost=4845.19..4850.64 rows=2179 width=47) (actual time=68.069..68.222 rows=695.00 loops=2)
                Sort Key: ((EXTRACT(year FROM age((CURRENT_DATE)::timestamp with time zone, (p.birth_date)::timestamp with time zone)))::integer)
                Sort Method: quicksort  Memory: 586kB
                Buffers: shared hit=1056
                Worker 0:  Sort Method: quicksort  Memory: 159kB
                -> Nested Loop  (cost=0.83..4724.37 rows=2179 width=47) (actual time=0.127..65.689 rows=3585.00 loops=2)
                   Join Filter: ((p.user_id <> me.user_id) AND ((pr.target_gender IS NULL) OR ((p.gender)::text = (pr.target_gender)::text)) AND ((pr.min_age IS NULL) OR (EXTR...
                   Rows Removed by Join Filter: 46415
                   Buffers: shared hit=1049
                   -> Parallel Seq Scan on "Profile" p  (cost=0.00..1619.24 rows=58824 width=31) (actual time=0.009..5.643 rows=50000.00 loops=2)
                      Buffers: shared hit=1031
                   -> Materialize  (cost=0.83..16.88 rows=1 width=30) (actual time=0.000..0.000 rows=1.00 loops=100000)
                      Storage: Memory  Maximum Storage: 17kB
                      Buffers: shared hit=18
                      -> Nested Loop  (cost=0.83..16.87 rows=1 width=30) (actual time=0.087..0.089 rows=1.00 loops=2)
                         Buffers: shared hit=18
                         -> Index Scan using user_email_uq on "User" me  (cost=0.42..8.44 rows=1 width=16) (actual time=0.055..0.056 rows=1.00 loops=2)
                            Index Cond: ((email)::text = 'user5@example.com'::text)
```

Початок плану запиту 3 з індексами (окремий запуск, час у вузлах трохи інший). Для Profile як і раніше Parallel Seq Scan, idx_profile_gender не використано

Порівняння

Запит	**Програма без / з індексами, мс**	**EXPLAIN Execution без / з, мс**	**Planning + Execution без / з, мс**	**Що змінилось**
1. Діапазон дат народження	86.91 / 2.21	86.053 / 0.553	86.419 / 2.550	швидше в 39 разів (EXPLAIN в 156)
2. Лайки за період, GROUP BY	115.77 / 84.09	113.420 / 81.826	114.233 / 83.225	швидше на 27 %
3. Кандидати для user5	114.73 / 111.31	114.306 / 107.783	115.875 / 109.305	майже без змін (3-6 %)

У запиті 1 після створення idx_profile_birth_date план змінився. Замість Parallel Seq Scan по всій таблиці Profile з сортуванням тепер Index Scan, а кількість прочитаних буферів зменшилась з 1273 до 255. Виграш великий, бо діапазон дат вибірковий, а ORDER BY birth_date з LIMIT 50 дозволяє брати рядки з індексу вже в потрібному порядку і прочитати близько 51 профілю замість 100 000. У програмі час впав з 86.91 до 2.21 мс. Planning Time з

індексом більший (1.997 проти 0.366 мс), мабуть, через перше звернення до нового індексу (в плані є read=2).

У запиті 2 індекс idx_like_timestamp теж використався: Parallel Seq Scan по Like замінився на Bitmap Index Scan. Але виграш менший, близько 27 %. Діапазон 2026-05-01 - 2026-06-30 охоплює близько 5 450 лайків, а решта плану лишилась такою самою: Profile читається повністю для з'єднання, а потім майже 5 450 разів береться User за первинним ключем. Тобто індекс прискорив тільки одну частину запиту.

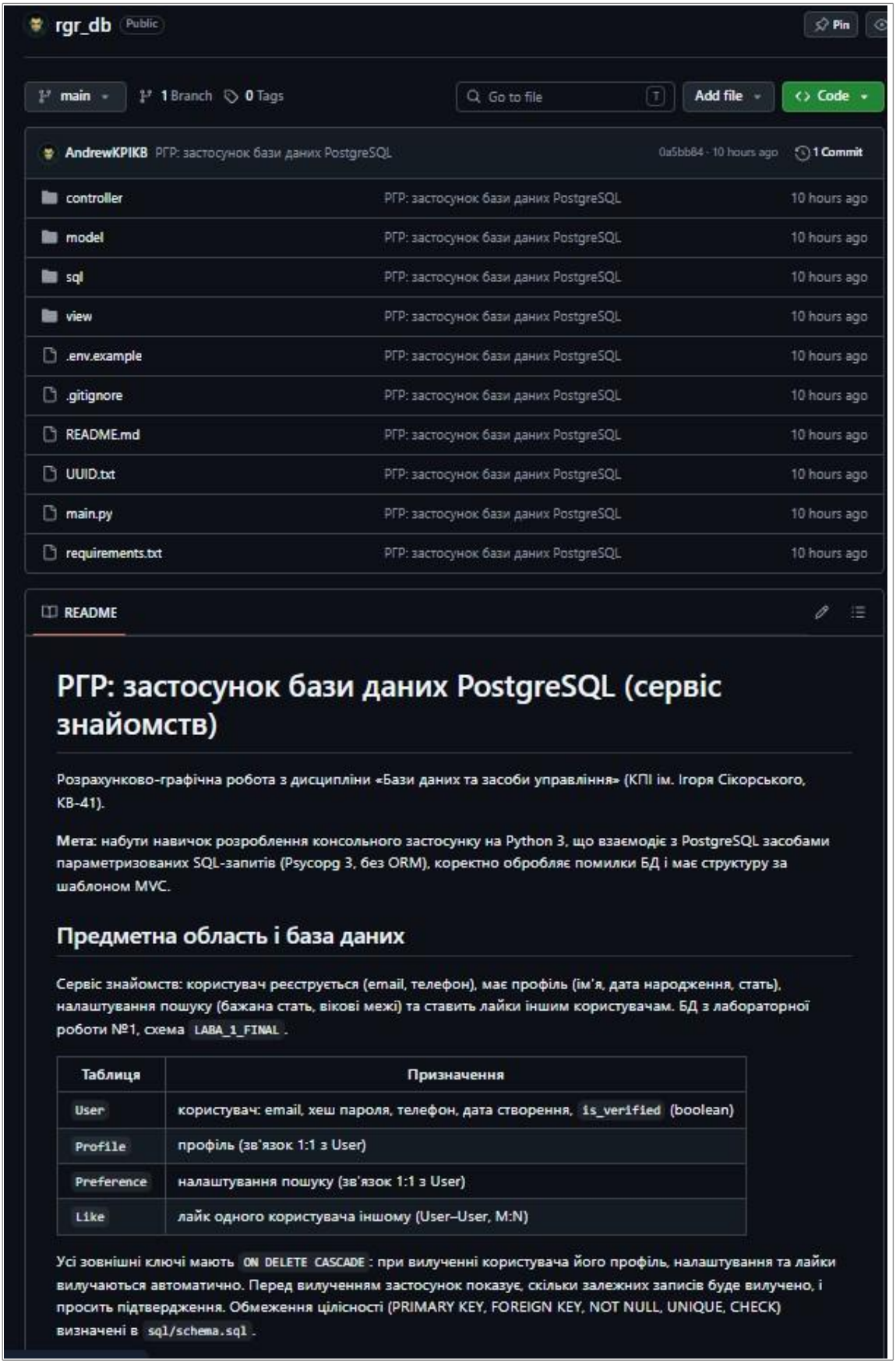
У запиті 3 час майже не змінився (114.73 і 111.31 мс у програмі, 114.306 і 107.783 мс за EXPLAIN). Така різниця це звичайний розкид між запусками. На початку плану з індексами видно, що таблиця Profile, як і раніше, читається через Parallel Seq Scan, тобто idx_profile_gender планувальник не використав. Це очікувано, бо індекс по статі має низьку селективність, а умови за віком обчислюються виразом над age(), який не можна проіндексувати. Без нових індексів запит і так використовує унікальні індекси схеми (user_email_uq, Preference_user_id_key, like_pair_uq, User_pkey), а таблицю Profile читає повністю.

Тобто індекси добре допомагають, коли умова вибіркова і стовпець використовується без обчислень (запит 1), частково допомагають, коли більшу частину часу запит витрачає на інші таблиці (запит 2), і не допомагають, коли умова не вибіркова або є обчислення над стовпцем (запит 3).

Пункт №4. Організація коду за шаблоном MVC

Код поділений на три пакети: model, view і controller. Model працює тільки з базою даних, View тільки виводить і приймає текст, Controller з'єднує їх. SQL є лише в model, ORM не використовується.

Компонент	Модулі	Що робить
Model	model/db.py model/tables.py model/crud.py model/generator.py model/search.py model/errors.py	Підключення до PostgreSQL; опис таблиць для CRUD; SQL перегляду, додавання, редагування та видалення; SQL генерації; пошукові запити з вимірюванням часу; переклад помилок PostgreSQL на повідомлення для користувача; commit і rollback.
View	view/console.py	Меню, введення, виведення таблиць і повідомлень. SQL і драйвера тут немає.
Controller	controller/main_controller.py controller/app.py controller/generation_controller.py controller/search_controller.py controller/inputs.py	Вибір пункту меню; сценарії CRUD, у тому числі підтвердження видалення; сценарії генерації та пошуку; перевірка введення (UUID, дата, true/false, число).
Точка входу	main.py	Створює підключення, запускає меню, закриває з'єднання.



Сторінка репозиторію на GitHub: код, SQL-скрипти, README.md, requirements.txt, .env.example

Висновки

У роботі зробив консольний застосунок на Python для бази даних PostgreSQL сайту знайомств. Через програму можна переглядати, додавати, редагувати і видаляти дані в усіх чотирьох таблицях. Неправильне введення програма відхиляє сама, а цілісність даних контролює база: порушення обмежень програма показує повідомленням українською, без системного тексту. Для зовнішніх ключів показав, як працюють CASCADE і RESTRICT, а перед видаленням програма показує, які записи будуть видалені.

Дані генеруються SQL-запитами, кількість вводить користувач. Згенерував по 100 000 записів у User, Profile, Preference і 99 998 у Like, перевірка показала 0 порушень обмежень. Зробив чотири параметризовані пошукові запити до кількох таблиць з різними умовами (ILIKE, діапазони, true/false, GROUP BY з HAVING). Значення передаються параметрами Psycopg, що перевірив спробами SQL-ін'єкції. Після кожного пошуку показується час виконання. Також перевірив, як індекси впливають на швидкість: запит 1 прискорився в 39 разів, запит 2 на 27 %, а запит 3 не змінився. Код розділений за MVC.

Додаток. SQL створення таблиць (sql/schema.sql)

```
CREATE SCHEMA IF NOT EXISTS "LABA_1_FINAL";
SET search_path TO "LABA_1_FINAL";

CREATE TABLE IF NOT EXISTS "User" (
    user_id       uuid         NOT NULL DEFAULT gen_random_uuid(),
    email         varchar(255) NOT NULL,
    password_hash varchar(255) NOT NULL,
    phone_number  varchar(30)  NOT NULL,
    created_at    date         NOT NULL DEFAULT CURRENT_DATE,
    is_verified   boolean      NOT NULL DEFAULT false,
    CONSTRAINT "User_pkey"    PRIMARY KEY (user_id),
    CONSTRAINT user_email_uq UNIQUE (email),
    CONSTRAINT user_phone_uq UNIQUE (phone_number),
    CONSTRAINT user_email_ck CHECK (email LIKE '%_@_%.__%')
);

CREATE TABLE IF NOT EXISTS "Profile" (
    profile_id uuid         NOT NULL DEFAULT gen_random_uuid(),
    user_id    uuid         NOT NULL,
    first_name varchar(100) NOT NULL,
    birth_date  date       NOT  NULL,
    gender     varchar(10)  NOT NULL,
    CONSTRAINT "Profile_pkey"         PRIMARY KEY (profile_id),
    CONSTRAINT "Profile_user_id_key" UNIQUE (user_id),
    CONSTRAINT profile_user_fk FOREIGN KEY (user_id)
        REFERENCES "User" (user_id) ON UPDATE NO ACTION ON DELETE CASCADE,
    CONSTRAINT profile_birth_ck  CHECK (birth_date > DATE '1900-01-01'),
    CONSTRAINT profile_gender_ck CHECK (gender IN ('male', 'female', 'other'))
);

CREATE TABLE IF NOT EXISTS "Preference" (
    preference_id uuid        NOT NULL DEFAULT gen_random_uuid(),
    user_id       uuid        NOT NULL,
    target_gender varchar(10),
    min_age       integer,
    max_age       integer,
    CONSTRAINT "Preference_pkey"         PRIMARY KEY (preference_id),
    CONSTRAINT "Preference_user_id_key" UNIQUE (user_id),
    CONSTRAINT preference_user_fk FOREIGN KEY (user_id)
        REFERENCES "User" (user_id) ON UPDATE NO ACTION ON DELETE CASCADE,
    CONSTRAINT pref_gender_ck  CHECK (target_gender IN ('male', 'female', 'other')),
    CONSTRAINT pref_min_age_ck CHECK (min_age >= 18),
    CONSTRAINT pref_max_age_ck CHECK (max_age <= 120),
    CONSTRAINT pref_range_ck   CHECK (min_age <= max_age)
);

CREATE TABLE IF NOT EXISTS "Like" (
    like_id      uuid NOT NULL DEFAULT gen_random_uuid(),
    user_id_to   uuid NOT NULL,
    user_id_from uuid NOT NULL,
    "timestamp"  date NOT NULL DEFAULT CURRENT_DATE,
    CONSTRAINT "Like_pkey" PRIMARY KEY (like_id),
    CONSTRAINT like_from_fk FOREIGN KEY (user_id_from)
        REFERENCES "User" (user_id) ON UPDATE NO ACTION ON DELETE CASCADE,
    CONSTRAINT like_to_fk FOREIGN KEY (user_id_to)
        REFERENCES "User" (user_id) ON UPDATE NO ACTION ON DELETE CASCADE,
    CONSTRAINT like_pair_uq UNIQUE (user_id_from, user_id_to),
    CONSTRAINT like_self_ck CHECK (user_id_from <> user_id_to)
);
```